CHAPTER 17

Integration of Ordinary Differential Equations

17.0 Introduction

Problems involving ordinary differential equations (ODEs) can always be reduced to the study of sets of first-order differential equations. For example the second-order equation

$$\frac{d^2y}{dx^2} + q(x)\frac{dy}{dx} = r(x) \tag{17.0.1}$$

can be rewritten as two first-order equations,

$$\begin{aligned} \frac{dy}{dx} &= z(x) \\ \frac{dz}{dx} &= r(x) - q(x)z(x) \end{aligned} \tag{17.0.2}$$

where z is a new variable. This exemplifies the procedure for an arbitrary ODE. The usual choice for the new variables is to let them be just derivatives of each other (and of the original variable). Occasionally, it is useful to incorporate into their definition some other factors in the equation, or some powers of the independent variable, for the purpose of mitigating singular behavior that could result in overflows or increased roundoff error. Let common sense be your guide: If you find that the original variables are smooth in a solution, while your auxiliary variables are doing crazy things, then figure out why and choose different auxiliary variables.

The generic problem in ordinary differential equations is thus reduced to the study of a set of N coupled *first-order* differential equations for the functions y_i, $i = 0, 1, \ldots, N-1$, having the general form

$$\frac{dy_i(x)}{dx} = f_i(x, y_0, \ldots, y_{N-1}), \qquad i = 0, \ldots, N-1 \tag{17.0.3}$$

where the functions f_i on the right-hand side are known.

A problem involving ODEs is not completely specified by its equations. Even more crucial in determining how to attack the problem numerically is the nature of

the problem's boundary conditions. Boundary conditions are algebraic conditions on the values of the functions y_i in (17.0.3). In general they can be satisfied at discrete specified points, but do not hold between those points, i.e., are not preserved automatically by the differential equations. Boundary conditions can be as simple as requiring that certain variables have certain numerical values, or as complicated as a set of nonlinear algebraic equations among the variables.

Usually, it is the nature of the boundary conditions that determines which numerical methods will be feasible. Boundary conditions divide into two broad categories.

- In *initial value problems* all the y_i are given at some starting value x_s, and it is desired to find the y_i's at some final point x_f, or at some discrete list of points (for example, at tabulated intervals).
- In *two-point boundary value problems*, on the other hand, boundary conditions are specified at more than one x. Typically, some of the conditions will be specified at x_s and the remainder at x_f.

This chapter will consider exclusively the initial value problem, deferring two-point boundary value problems, which are generally more difficult, to Chapter 18.

The underlying idea of any routine for solving the initial value problem is always this: Rewrite the dy's and dx's in (17.0.3) as finite steps Δy and Δx, and multiply the equations by Δx. This gives algebraic formulas for the change in the functions when the independent variable x is "stepped" by one "stepsize" Δx. In the limit of making the stepsize very small, a good approximation to the underlying differential equation is achieved. Literal implementation of this procedure results in *Euler's method* (equation 17.1.1, below), which is, however, *not* recommended for any practical use. Euler's method is conceptually important, however; one way or another, practical methods all come down to this same idea: Add small increments to your functions corresponding to derivatives (right-hand sides of the equations) multiplied by stepsizes.

In this chapter we consider three major types of practical numerical methods for solving initial value problems for ODEs:

- Runge-Kutta methods
- Richardson extrapolation and its particular implementation as the Bulirsch-Stoer method
- predictor-corrector methods, also known as multistep methods.

A brief description of each of these types follows.

1. *Runge-Kutta* methods propagate a solution over an interval by combining the information from several Euler-style steps (each involving one evaluation of the right-hand f's), and then using the information obtained to match a Taylor series expansion up to some higher order.

2. *Richardson extrapolation* uses the powerful idea of extrapolating a computed result to the value that *would* have been obtained if the stepsize had been very much smaller than it actually was. In particular, extrapolation to zero stepsize is the desired goal. The first practical ODE integrator that implemented this idea was developed by Bulirsch and Stoer, and so extrapolation methods are often called Bulirsch-Stoer methods.

3. *Predictor-corrector* methods or *multistep methods* store the solution along the way, and use those results to extrapolate the solution one step advanced; they

then correct the extrapolation using derivative information at the new point. These are best for very smooth functions.

Runge-Kutta used to be what you used when (i) you didn't know any better, or (ii) you had an intransigent problem where Bulirsch-Stoer was failing, or (iii) you had a trivial problem where computational efficiency was of no concern. However, advances in Runge-Kutta methods, particularly the development of higher-order methods, have made Runge-Kutta competitive with the other methods in many cases. Runge-Kutta succeeds virtually always; it is usually the fastest method when evaluating f_i is cheap and the accuracy requirement is not ultra-stringent ($\lesssim 10^{-10}$), or in general when moderate accuracy ($\lesssim 10^{-5}$) is required. Predictor-corrector methods have a relatively high overhead and so come into their own only when evaluating f_i is expensive. However, for many smooth problems, they are computationally more efficient than Runge-Kutta. In recent years, Bulirsch-Stoer has been replacing predictor-corrector in many applications, but it is too soon to say that predictor-corrector is dominated in all cases. However, it appears that only rather sophisticated predictor-corrector routines are competitive. Accordingly, we have chosen *not* to give an implementation of predictor-corrector in this book. We discuss predictor-corrector further in §17.6, so that you can use a packaged routine knowledgeably should you encounter a suitable problem. In our experience, the relatively simple Runge-Kutta and Bulirsch-Stoer routines we give are adequate for most problems.

Each of the three types of methods can be organized to monitor internal consistency. This allows numerical errors, which are inevitably introduced into the solution, to be controlled by automatic (*adaptive*) changing of the fundamental stepsize. We always recommend that adaptive stepsize control be implemented, and we will do so below.

In general, all three types of methods can be applied to any initial value problem. Each comes with its own set of debits and credits that must be understood before it is used.

Section 17.5 of this chapter treats the subject of *stiff equations*, relevant both to ordinary differential equations and also to partial differential equations (Chapter 20).

17.0.1 Organization of the Routines in This Chapter

We have organized the routines in this chapter into three nested levels, enabling modularity and sharing common code wherever possible.

The highest level is the *driver* object, which starts and stops the integration, stores intermediate results, and generally acts as an interface with the user. There is nothing canonical about our driver object, `Odeint`. You should consider it to be an example, and you can customize it for your particular application.

The next level down is a *stepper* object. The stepper oversees the actual incrementing of the independent variable x. It knows how to call the underlying *algorithm* routine. It may reject the result, set a smaller stepsize, and call the algorithm routine again, until compatibility with a predetermined accuracy criterion has been achieved. The stepper's fundamental task is to take the largest stepsize consistent with specified performance. Only when this is accomplished does the true power of an algorithm come to light.

All our steppers are derived from a base object called `StepperBase`: `StepperDopr5` and `StepperDopr853` (two Runge-Kutta routines), `StepperBS` and `StepperStoerm` (two Bulirsch-Stoer routines), and `StepperRoss` and `StepperSIE`

(for so-called stiff equations).

Standing apart from the stepper, but interacting with it at the same level, is an `Output` object. This is basically a container into which the stepper writes the output of the integration, but it has some intelligence of its own: It can save, or not save, intermediate results according to several different prescriptions that are specified by its constructor. In particular, it has the option to provide so-called dense output, that is, output at user-specified intermediate points without loss of efficiency.

The lowest or "nitty-gritty" level is the piece we call the *algorithm* routine. This implements the basic formulas of the method, starts with dependent variables y_i at x, and calculates new values of the dependent variables at the value $x + h$. The algorithm routine also yields some information about the quality of the solution after the step. The routine is dumb, however, in that it is unable to make any adaptive decision about whether the solution is of acceptable quality. Each algorithm routine is implemented as a member function `dy()` in its corresponding stepper object.

17.0.2 The Odeint Object

It is a real time saver to have a single high-level interface to what are otherwise quite diverse methods. We use the `Odeint` driver for a variety of problems, notably including garden-variety ODEs or sets of ODEs, and definite integrals (augmenting the methods of Chapter 4). The `Odeint` driver is templated on the stepper. This means that you can usually change from one ODE method to another in just a few keystrokes. For example, changing from the Dormand-Prince fifth-order Runge-Kutta method to Bulirsch-Stoer is as simple as changing the template parameter from `StepperDopr5` to `StepperBS`.

The `Odeint` constructor simply initializes a bunch of things, including a call to the stepper constructor. The meat is in the `integrate` routine, which repeatedly invokes the `step` routine of the stepper to advance the solution from x_1 to x_2. It also calls the functions of the `Output` object to save the results at appropriate points.

odeint.h

```
template<class Stepper>
struct Odeint {
```

Driver for ODE solvers with adaptive stepsize control. The template parameter should be one of the derived classes of `StepperBase` defining a particular integration algorithm.

```
    static const Int MAXSTP=50000;
    Doub EPS;
    Int nok;
    Int nbad;
    Int nvar;
    Doub x1,x2,hmin;
    bool dense;
    VecDoub y,dydx;
    VecDoub &ystart;
    Output &out;
    typename Stepper::Dtype &derivs;
    Stepper s;
    Int nstp;
    Doub x,h;
    Odeint(VecDoub_IO &ystartt,const Doub xx1,const Doub xx2,
        const Doub atol,const Doub rtol,const Doub h1,
        const Doub hminn,Output &outt,typename Stepper::Dtype &derivss);
```

Take at most `MAXSTP` steps.

`true` if dense output requested by `out`.

Get the type of `derivs` from the stepper.

Constructor sets everything up. The routine integrates starting values `ystart[0..nvar-1]` from `xx1` to `xx2` with absolute tolerance `atol` and relative tolerance `rtol`. The quantity `h1` should be set as a guessed first stepsize, `hmin` as the minimum allowed stepsize (can be zero). An `Output` object `out` should be input to control the saving of intermediate values.

On output, nok and nbad are the number of good and bad (but retried and fixed) steps taken, and ystart is replaced by values at the end of the integration interval. derivs is the user-supplied routine (function or functor) for calculating the right-hand side derivative.

```
    void integrate();                                   Does the actual integration.
};

template<class Stepper>
Odeint<Stepper>::Odeint(VecDoub_IO &ystartt, const Doub xx1, const Doub xx2,
    const Doub atol, const Doub rtol, const Doub h1, const Doub hminn,
    Output &outt,typename Stepper::Dtype &derivss) : nvar(ystartt.size()),
    y(nvar),dydx(nvar),ystart(ystartt),x(xx1),nok(0),nbad(0),
    x1(xx1),x2(xx2),hmin(hminn),dense(outt.dense),out(outt),derivs(derivss),
    s(y,dydx,x,atol,rtol,dense) {
    EPS=numeric_limits<Doub>::epsilon();
    h=SIGN(h1,x2-x1);
    for (Int i=0;i<nvar;i++) y[i]=ystart[i];
    out.init(s.neqn,x1,x2);
}

template<class Stepper>
void Odeint<Stepper>::integrate() {
    derivs(x,y,dydx);
    if (dense)                                          Store initial values.
        out.out(-1,x,y,s,h);
    else
        out.save(x,y);
    for (nstp=0;nstp<MAXSTP;nstp++) {
        if ((x+h*1.0001-x2)*(x2-x1) > 0.0)
            h=x2-x;                                     If stepsize can overshoot, decrease.
        s.step(h,derivs);                               Take a step.
        if (s.hdid == h) ++nok; else ++nbad;
        if (dense)
            out.out(nstp,x,y,s,s.hdid);
        else
            out.save(x,y);
        if ((x-x2)*(x2-x1) >= 0.0) {                    Are we done?
            for (Int i=0;i<nvar;i++) ystart[i]=y[i];        Update ystart.
            if (out.kmax > 0 && abs(out.xsave[out.count-1]-x2) > 100.0*abs(x2)*EPS)
                out.save(x,y);                          Make sure last step gets saved.
            return;                                     Normal exit.
        }
        if (abs(s.hnext) <= hmin) throw("Step size too small in Odeint");
        h=s.hnext;
    }
    throw("Too many steps in routine Odeint");
}
```

The Odeint object doesn't know in advance which specific stepper object it will be instantiated with. It does, however, rely on the fact that the stepper object will be derived from, and thus have the methods in, this StepperBase object, which serves as the base class for all subsequent ODE algorithms in this chapter:

```
struct StepperBase {                                                    stepper.h
Base class for all ODE algorithms.
    Doub &x;
    Doub xold;                          Used for dense output.
    VecDoub &y,&dydx;
    Doub atol,rtol;
    bool dense;
    Doub hdid;                          Actual stepsize accomplished by the step routine.
    Doub hnext;                         Stepsize predicted by the controller for the next step.
```

```
    Doub EPS;
    Int n,neqn;                          neqn = n except for StepperStoerm.
    VecDoub yout,yerr;                   New value of y and error estimate.
    StepperBase(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx, const Doub atoll,
        const Doub rtoll, bool dens) : x(xx),y(yy),dydx(dydxx),atol(atoll),
        rtol(rtoll),dense(dens),n(y.size()),neqn(n),yout(n),yerr(n) {}
        Input to the constructor are the dependent variable vector y[0..n-1] and its derivative
        dydx[0..n-1] at the starting value of the independent variable x. Also input are the
        absolute and relative tolerances, atol and rtol, and the boolean dense, which is true
        if dense output is required.
};
```

17.0.3 The Output Object

Output is controlled by the various constructors in the `Output` structure. The default constructor, with no arguments, suppresses all output. The constructor with argument `nsave` provides *dense output* provided `nsave` > 0. This means output at values of x of your choosing, not necessarily the natural places that the stepper method would land. The output points are `nsave` $+ 1$ uniformly spaced points including `x1` and `x2`. If `nsave` ≤ 0, output is saved at every integration step, that is, only at the points where the stepper happens to land. While most of your needs should be met by these options, you should find it easy to modify `Output` for your particular application.

odeint.h

```
struct Output {
```
Structure for output from ODE solver such as `odeint`.
```
    Int kmax;                            Current capacity of storage arrays.
    Int nvar;
    Int nsave;                           Number of intervals to save at for dense output.
    bool dense;                          true if dense output requested.
    Int count;                           Number of values actually saved.
    Doub x1,x2,xout,dxout;
    VecDoub xsave;                       Results stored in the vector xsave[0..count-1] and the
    MatDoub ysave;                           matrix ysave[0..nvar-1][0..count-1].
    Output() : kmax(-1),dense(false),count(0) {}
```
Default constructor gives no output.
```
    Output(const Int nsavee) : kmax(500),nsave(nsavee),count(0),xsave(kmax) {
```
Constructor provides dense output at `nsave` equally spaced intervals. If `nsave` ≤ 0, output is saved only at the actual integration steps.
```
        dense = nsave > 0 ? true : false;
    }
    void init(const Int neqn, const Doub xlo, const Doub xhi) {
```
Called by `Odeint` constructor, which passes `neqn`, the number of equations, `xlo`, the starting point of the integration, and `xhi`, the ending point.
```
        nvar=neqn;
        if (kmax == -1) return;
        ysave.resize(nvar,kmax);
        if (dense) {
            x1=xlo;
            x2=xhi;
            xout=x1;
            dxout=(x2-x1)/nsave;
        }
    }
    void resize() {
```
Resize storage arrays by a factor of two, keeping saved data.
```
        Int kold=kmax;
        kmax *= 2;
        VecDoub tempvec(xsave);
```

```
        xsave.resize(kmax);
        for (Int k=0; k<kold; k++)
            xsave[k]=tempvec[k];
        MatDoub tempmat(ysave);
        ysave.resize(nvar,kmax);
        for (Int i=0; i<nvar; i++)
            for (Int k=0; k<kold; k++)
                ysave[i][k]=tempmat[i][k];
    }
    template <class Stepper>
    void save_dense(Stepper &s, const Doub xout, const Doub h) {
    Invokes dense_out function of stepper routine to produce output at xout. Normally called
    by out rather than directly. Assumes that xout is between xold and xold+h, where the
    stepper must keep track of xold, the location of the previous step, and x=xold+h, the
    current step.
        if (count == kmax) resize();
        for (Int i=0;i<nvar;i++)
            ysave[i][count]=s.dense_out(i,xout,h);
        xsave[count++]=xout;
    }
    void save(const Doub x, VecDoub_I &y) {
    Saves values of current x and y.
        if (kmax <= 0) return;
        if (count == kmax) resize();
        for (Int i=0;i<nvar;i++)
            ysave[i][count]=y[i];
        xsave[count++]=x;
    }
    template <class Stepper>
    void out(const Int nstp,const Doub x,VecDoub_I &y,Stepper &s,const Doub h) {
    Typically called by Odeint to produce dense output. Input variables are nstp, the current
    step number, the current values of x and y, the stepper s, and the stepsize h. A call with
    nstp=-1 saves the initial values. The routine checks whether x is greater than the desired
    output point xout. If so, it calls save_dense.
        if (!dense)
            throw("dense output not set in Output!");
        if (nstp == -1) {
            save(x,y);
            xout += dxout;
        } else {
            while ((x-xout)*(x2-x1) > 0.0) {
                save_dense(s,xout,h);
                xout += dxout;
            }
        }
    }
};
```

17.0.4 A Quick-Start Example

Before we dive deep into the pros and cons of the different stepper types (the meat of this chapter), let's see how to code the solution of an actual problem. Suppose we want to solve Van der Pol's equation, which when written in first-order form is

$$\begin{aligned} y_0' &= y_1 \\ y_1' &= [(1 - y_0^2)y_1 - y_0]/\epsilon \end{aligned} \tag{17.0.4}$$

First encapsulate (17.0.4) in a functor (see §1.3.3). Using a functor instead of a bare function gives you the opportunity to pass other information to the function,

such as the values of fixed parameters. Every stepper class in this chapter is accordingly templated on the type of the functor defining the right-hand side derivatives. For our example, the right-hand side functor looks like:

```
struct rhs_van {
    Doub eps;
    rhs_van(Doub epss) : eps(epss) {}
    void operator() (const Doub x, VecDoub_I &y, VecDoub_O &dydx) {
        dydx[0]= y[1];
        dydx[1]=((1.0-y[0]*y[0])*y[1]-y[0])/eps;
    }
};
```

The key thing is the line beginning `void operator()`: It *always* should have this form, with the definition of `dydx` following. Here we have chosen to specify ϵ as a parameter in the constructor so that the main program can easily pass a specific value to the right-hand side. Alternatively, you could have omitted the constructor, relying on the compiler-supplied default constructor, and hard-coded a value of ϵ in the routine. Note, of course, that there is nothing special about the name `rhs_van`.

We will integrate from 0 to 2 with initial conditions $y_0 = 2$, $y_1 = 0$ and with $\epsilon = 10^{-3}$. Then your main program will have declarations like the following:

```
const Int nvar=2;
const Doub atol=1.0e-3, rtol=atol, h1=0.01, hmin=0.0, x1=0.0, x2=2.0;
VecDoub ystart(nvar);
ystart[0]=2.0;
ystart[1]=0.0;
Output out(20);                          Dense output at 20 points plus x1.
rhs_van d(1.0e-3);                       Declare d as a rhs_van object.
Odeint<StepperDopr5<rhs_van> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
ode.integrate();
```

Note how the `Odeint` object is templated on the stepper, which in turn is templated on the derivative object, `rhs_van` in this case. The space between the two closing angle brackets is necessary; otherwise the compiler parses >> as the right-shift operator!

The number of good steps taken is available in `ode.nok` and the number of bad steps in `ode.nbad`. The output, which is equally spaced, can be printed by statements like

```
for (Int i=0;i<out.count;i++)
    cout << out.xsave[i] << " " << out.ysave[0][i] << " " <<
        out.ysave[1][i] << endl;
```

You can alternatively save output at the actual integration steps by the declaration

```
Output out(-1);
```

or suppress all saving of output with

```
Output out;
```

In this case, the solution values at the endpoint are available in `ystart[0]` and `ystart[1]`, overwriting the starting values.

CITED REFERENCES AND FURTHER READING:

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall).

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 5.

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 7.

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer)

Hairer, E., Nørsett, S.P., and Wanner, G. 1996, *Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems*, 2nd ed. (New York: Springer)

Lambert, J. 1973, *Computational Methods in Ordinary Differential Equations* (New York: Wiley).

Lapidus, L., and Seinfeld, J. 1971, *Numerical Solution of Ordinary Differential Equations* (New York: Academic Press).

17.1 Runge-Kutta Method

The formula for the Euler method is

$$y_{n+1} = y_n + hf(x_n, y_n) \tag{17.1.1}$$

which advances a solution from x_n to $x_{n+1} \equiv x_n + h$. The formula is unsymmetrical: It advances the solution through an interval h, but uses derivative information only at the beginning of that interval (see Figure 17.1.1). That means (and you can verify by expansion in power series) that the step's error is only one power of h smaller than the correction, i.e., $O(h^2)$ added to (17.1.1).

There are several reasons that Euler's method is not recommended for practical use, among them, (i) the method is not very accurate when compared to other, fancier, methods run at the equivalent stepsize, and (ii) neither is it very stable (see §17.5 below).

Consider, however, the use of a step like (17.1.1) to take a "trial" step to the midpoint of the interval. Then use the values of both x and y at that midpoint to compute the "real" step across the whole interval. Figure 17.1.2 illustrates the idea. In equations,

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf\left(x_n + \tfrac{1}{2}h, y_n + \tfrac{1}{2}k_1\right) \\ y_{n+1} &= y_n + k_2 + O(h^3) \end{aligned} \tag{17.1.2}$$

As indicated in the error term, this symmetrization cancels out the first-order error term, making the method *second order*. [A method is conventionally called nth order if its error term is $O(h^{n+1})$.] In fact, (17.1.2) is called the *second-order Runge-Kutta* or *midpoint* method.

We needn't stop there. There are many ways to evaluate the right-hand side $f(x, y)$ that all agree to first order, but that have different coefficients of higher-order error terms. Adding up the right combination of these, we can eliminate the error terms order by order. That is the basic idea of the Runge-Kutta method. Abramowitz and Stegun [1] and Gear [2] give various specific formulas that derive from this basic idea. By far the most often used is the classical *fourth-order Runge-Kutta formula*,

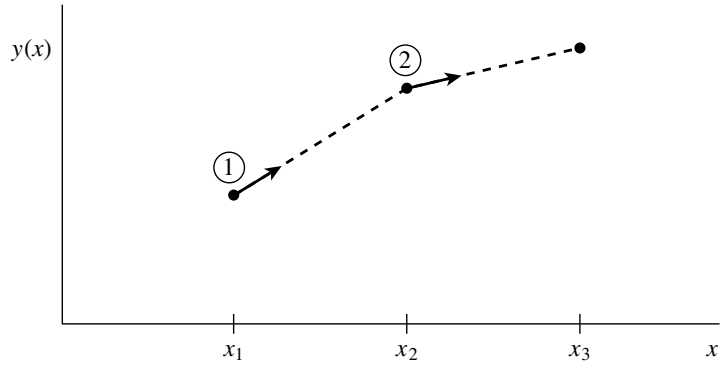


Figure 17.1.1. Euler's method. In this simplest (and least accurate) method for integrating an ODE, the derivative at the starting point of each interval is extrapolated to find the next function value. The method has first-order accuracy.

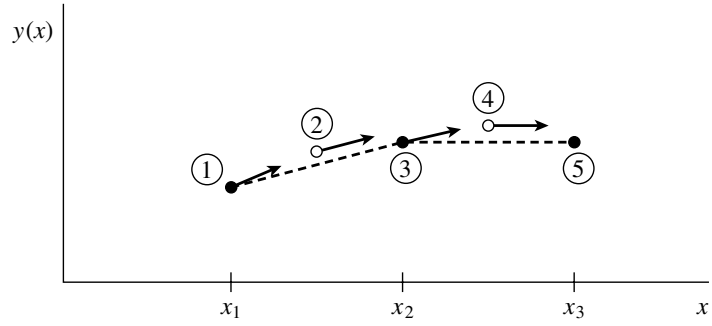


Figure 17.1.2. Midpoint method. Second-order accuracy is obtained by using the initial derivative at each step to find a point halfway across the interval, then using the midpoint derivative across the full width of the interval. In the figure, filled dots represent final function values, while open dots represent function values that are discarded once their derivatives have been calculated and used.

which has a certain sleekness of organization about it:

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf(x_n + \tfrac{1}{2}h, y_n + \tfrac{1}{2}k_1) \\ k_3 &= hf(x_n + \tfrac{1}{2}h, y_n + \tfrac{1}{2}k_2) \\ k_4 &= hf(x_n + h, y_n + k_3) \\ y_{n+1} &= y_n + \tfrac{1}{6}k_1 + \tfrac{1}{3}k_2 + \tfrac{1}{3}k_3 + \tfrac{1}{6}k_4 + O(h^5) \end{aligned} \tag{17.1.3}$$

The fourth-order Runge-Kutta method requires four evaluations of the right-hand side per step h (see Figure 17.1.3). This will be superior to the midpoint method (17.1.2) *if* at least twice as large a step is possible with (17.1.3) for the same accuracy. Is that so? The answer is: often, perhaps even usually, but surely not always! This takes us back to a central theme, namely that *high order* does not always mean *high accuracy*. The statement "fourth-order Runge-Kutta is generally superior to second-order" is a true one, but as much a statement about the kind of problems that people solve as a statement about strict mathematics.

For many scientific users, fourth-order Runge-Kutta is not just the first word

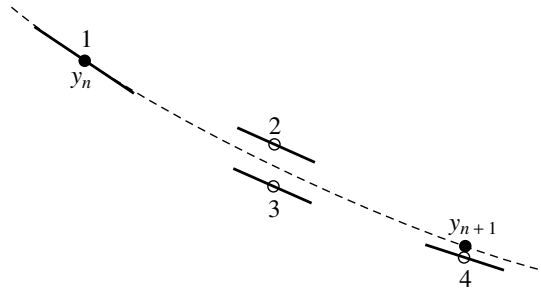


Figure 17.1.3. Fourth-order Runge-Kutta method. In each step the derivative is evaluated four times: once at the initial point, twice at trial midpoints, and once at a trial endpoint. From these derivatives the final function value (shown as a filled dot) is calculated. (See text for details.)

on ODE integrators, but the last word as well. In fact, you can get pretty far on this old workhorse, especially if you combine it with an adaptive stepsize algorithm. Keep in mind, however, that the old workhorse's last trip may well be to take you to the poorhouse: Newer Runge-Kutta methods are *much* more efficient, and Bulirsch-Stoer or predictor-corrector methods can be even more efficient for problems where very high accuracy is a requirement. Those methods are the high-strung racehorses. Runge-Kutta is for ploughing the fields. However, even the old workhorse is more nimble with new horseshoes. In §17.2 we will give a modern implementation of a Runge-Kutta method that is quite competitive as long as very high accuracy is not required. An excellent discussion of the pitfalls in constructing a good Runge-Kutta code is given in [3].

Here is the routine `rk4` for carrying out one classical Runge-Kutta step on a set of `n` differential equations. This routine is completely separate from the various stepper routines introduced in the previous section and given in the rest of the chapter. It is meant for only the most trivial applications. You input the values of the independent variables, and you get out new values that are stepped by a stepsize `h` (which can be positive or negative). You will notice that the routine requires you to supply not only function `derivs` for calculating the right-hand side, but also values of the derivatives at the starting point. Why not let the routine call `derivs` for this first value? The answer will become clear only in the next section, but in brief is this: This call may not be your only one with these starting conditions. You may have taken a previous step with too large a stepsize, and this is your replacement. In that case, you do not want to call `derivs` unnecessarily at the start. Note that the routine that follows has, therefore, only three calls to `derivs`.

rk4.h

```
void rk4(VecDoub_I &y, VecDoub_I &dydx, const Doub x, const Doub h,
    VecDoub_O &yout, void derivs(const Doub, VecDoub_I &, VecDoub_O &))
```

Given values for the variables `y[0..n-1]` and their derivatives `dydx[0..n-1]` known at `x`, use the fourth-order Runge-Kutta method to advance the solution over an interval `h` and return the incremented variables as `yout[0..n-1]`. The user supplies the routine `derivs(x,y,dydx)`, which returns derivatives `dydx` at `x`.

```
{
    Int n=y.size();
    VecDoub dym(n),dyt(n),yt(n);
    Doub hh=h*0.5;
    Doub h6=h/6.0;
    Doub xh=x+hh;
```

```
    for (Int i=0;i<n;i++) yt[i]=y[i]+hh*dydx[i];
    derivs(xh,yt,dyt);
    for (Int i=0;i<n;i++) yt[i]=y[i]+hh*dyt[i];
    derivs(xh,yt,dym);
    for (Int i=0;i<n;i++) {
        yt[i]=y[i]+h*dym[i];
        dym[i] += dyt[i];
    }
    derivs(x+h,yt,dyt);
    for (Int i=0;i<n;i++)
        yout[i]=y[i]+h6*(dydx[i]+dyt[i]+2.0*dym[i]);
}
```

First step.
Second step.

Third step.

Fourth step.
Accumulate increments with proper weights.

The Runge-Kutta method treats every step in a sequence of steps in an identical manner. Prior behavior of a solution is not used in its propagation. This is mathematically proper, since any point along the trajectory of an ordinary differential equation can serve as an initial point. The fact that all steps are treated identically also makes it easy to incorporate Runge-Kutta into relatively simple "driver" schemes.

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`, §25.5.[1]

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 2.[2]

Shampine, L.F., and Watts, H.A. 1977, "The Art of Writing a Runge-Kutta Code, Part I," in *Mathematical Software III*, J.R. Rice, ed. (New York: Academic Press), pp. 257–275; 1979, "The Art of Writing a Runge-Kutta Code. II," *Applied Mathematics and Computation*, vol. 5, pp. 93–121.[3]

17.2 Adaptive Stepsize Control for Runge-Kutta

A good ODE integrator should exert some adaptive control over its own progress, making frequent changes in its stepsize. Usually the purpose of this adaptive stepsize control is to achieve some predetermined accuracy in the solution with minimum computational effort. Many small steps should tiptoe through treacherous terrain, while a few great strides should speed through smooth uninteresting countryside. The resulting gains in efficiency are not mere tens of percents or factors of two; they can sometimes be factors of ten, a hundred, or more. Sometimes accuracy may be demanded not directly in the solution itself, but in some related conserved quantity that can be monitored.

Implementation of adaptive stepsize control requires that the stepping algorithm signal information about its performance, most important, an estimate of its truncation error. In this section we will learn how such information can be obtained. Obviously, the calculation of this information will add to the computational overhead, but the investment will generally be repaid handsomely.

With fourth-order Runge-Kutta, the most straightforward technique by far is *step doubling* (see, e.g., [1]). We take each step twice, once as a full step, then,

independently, as two half-steps (see Figure 17.2.1). How much overhead is this, say in terms of the number of evaluations of the right-hand sides? Each of the three separate Runge-Kutta steps in the procedure requires 4 evaluations, but the single and double sequences share a starting point, so the total is 11. This is to be compared not to 4, but to 8 (the two half-steps), since — stepsize control aside — we are achieving the accuracy of the smaller (half-) stepsize. The overhead cost is therefore a factor 1.375. What does it buy us?

Let us denote the exact solution for an advance from x to $x + 2h$ by $y(x + 2h)$ and the two approximate solutions by y_1 (one step $2h$) and y_2 (two steps each of size h). Since the basic method is fourth order, the true solution and the two numerical approximations are related by

$$\begin{aligned} y(x + 2h) &= y_1 + (2h)^5\phi + O(h^6) + \dots \\ y(x + 2h) &= y_2 + 2(h^5)\phi + O(h^6) + \dots \end{aligned} \tag{17.2.1}$$

where, to order h^5, the value ϕ remains constant over the step. [Taylor series expansion tells us the ϕ is a number whose order of magnitude is $y^{(5)}(x)/5!$.] The first expression in (17.2.1) involves $(2h)^5$ since the stepsize is $2h$, while the second expression involves $2(h^5)$ since the error on each step is $h^5\phi$. The difference between the two numerical estimates is a convenient indicator of truncation error,

$$\Delta \equiv y_2 - y_1 \tag{17.2.2}$$

It is this difference that we shall endeavor to keep to a desired degree of accuracy, neither too large nor too small. We do this by adjusting h.

It might also occur to you that, ignoring terms of order h^6 and higher, we can solve the two equations in (17.2.1) to improve our numerical estimate of the true solution $y(x + 2h)$, namely,

$$y(x + 2h) = y_2 + \frac{\Delta}{15} + O(h^6) \tag{17.2.3}$$

This estimate is accurate to *fifth order*, one order higher than the original Runge-Kutta steps (Richardson extrapolation again!). However, we can't have our cake and eat it too: (17.2.3) may be fifth-order accurate, but we have no way of monitoring *its* truncation error. Higher order is not always higher accuracy! Use of (17.2.3) rarely does harm, but we have no way of directly knowing whether it is doing any good. Therefore we should use Δ as the error estimate and take as "gravy" any additional accuracy gain derived from (17.2.3). In the technical literature, use of a procedure like (17.2.3) is called "local extrapolation."

Step doubling has been superseded by a more efficient stepsize adjustment algorithm based on *embedded Runge-Kutta formulas*, originally invented by Merson and popularized in a method of Fehlberg. An interesting fact about Runge-Kutta formulas is that for orders M higher than four, more than M function evaluations are required. This accounts for the popularity of the classical fourth-order method: It seems to give the most bang for the buck. However, Fehlberg discovered a fifth-order method with six function evaluations where another combination of the six functions gives a fourth-order method. The difference between the two estimates of $y(x + h)$ can then be used as an estimate of the truncation error to adjust the

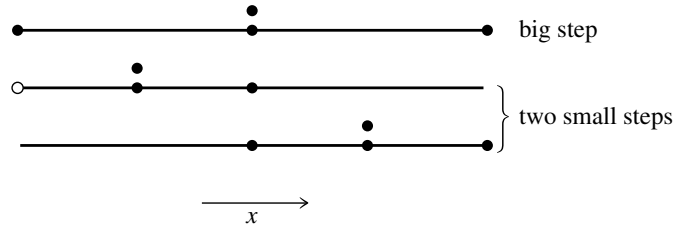


Figure 17.2.1. Step doubling as a means for adaptive stepsize control in fourth-order Runge-Kutta. Points where the derivative is evaluated are shown as filled circles. The open circle represents the same derivatives as the filled circle immediately above it, so the total number of evaluations is 11 per two steps. Comparing the accuracy of the big step with the two small steps gives a criterion for adjusting the stepsize on the next step, or for rejecting the current step as inaccurate.

stepsize. Since Fehlberg's original formula, many other embedded Runge-Kutta formulas have been found.

As an aside, the general question of how many function evaluations are required for a Runge-Kutta method of a given order is still open. Order 5 requires 6 function evaluations, order 6 requires 7, order 7 requires 9, order 8 requires 11. It is known that for order $M \geq 8$, at least $M + 3$ evaluations are required. The highest order explicitly constructed method so far is order 10, with 17 evaluations. The calculation of the coefficients of these high-order methods is very complicated.

We will spend most of this section setting up an efficient fifth-order Runge-Kutta method, coded in the routine `StepperDopr5`. This will allow us to explore the various issues that have to be dealt with in any Runge-Kutta scheme. However, ultimately you should not use this routine except for low accuracy requirements ($\lesssim 10^{-3}$) or trivial problems. Use the more efficient higher-order Runge-Kutta code `StepperDopr853` or the Bulirsch-Stoer code `StepperBS`.

The general form of a fifth-order Runge-Kutta formula is

$$\begin{aligned} k_1 &= hf(x_n, y_n) \\ k_2 &= hf(x_n + c_2 h, y_n + a_{21} k_1) \\ &\cdots \\ k_6 &= hf(x_n + c_6 h, y_n + a_{61} k_1 + \cdots + a_{65} k_5) \\ y_{n+1} &= y_n + b_1 k_1 + b_2 k_2 + b_3 k_3 + b_4 k_4 + b_5 k_5 + b_6 k_6 + O(h^6) \end{aligned} \tag{17.2.4}$$

The embedded fourth-order formula is

$$y^*_{n+1} = y_n + b^*_1 k_1 + b^*_2 k_2 + b^*_3 k_3 + b^*_4 k_4 + b^*_5 k_5 + b^*_6 k_6 + O(h^5) \tag{17.2.5}$$

and so the error estimate is

$$\Delta \equiv y_{n+1} - y^*_{n+1} = \sum_{i=1}^{6} (b_i - b^*_i) k_i \tag{17.2.6}$$

The particular values of the various constants that we favor are those found by Dormand and Prince [2] and given in the table on the next page. These give a more efficient method than Fehlberg's original values, with better error properties.

We said that the Dormand-Prince method needs six function evaluations per step, yet the table on the next page shows seven and the sums in equations (17.2.5) and (17.2.6) should really go up to $i = 7$. What's going on? The idea is to use

Dormand-Prince 5(4) Parameters for Embedded Runga-Kutta Method

i	c_i	a_{ij}						b_i	b_i^*
1								$\frac{35}{384}$	$\frac{5179}{57600}$
2	$\frac{1}{5}$	$\frac{1}{5}$						0	0
3	$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$					$\frac{500}{1113}$	$\frac{7571}{16695}$
4	$\frac{4}{5}$	$\frac{44}{45}$	$-\frac{56}{15}$	$\frac{32}{9}$				$\frac{125}{192}$	$\frac{393}{640}$
5	$\frac{8}{9}$	$\frac{19372}{6561}$	$-\frac{25360}{2187}$	$\frac{64448}{6561}$	$-\frac{212}{729}$			$-\frac{2187}{6784}$	$-\frac{92097}{339200}$
6	1	$\frac{9017}{3168}$	$-\frac{355}{33}$	$\frac{46732}{5247}$	$\frac{49}{176}$	$-\frac{5103}{18656}$		$\frac{11}{84}$	$\frac{187}{2100}$
7	1	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	0	$\frac{1}{40}$
$j =$		1	2	3	4	5	6		

y_{n+1} itself to provide a seventh stage. Because $f(x_n + h, y_{n+1})$ has to be evaluated anyway to start the next step, this costs nothing (unless the step is rejected because the error is too big). This trick is called FSAL (first-same-as-last). You can see in the table that the coefficients in the last row are the same as the b_i column.

Now that we know, at least approximately, what our error is, we need to consider how to keep it within desired bounds. We require

$$|\Delta| = |y_{n+1} - y^*_{n+1}| \le \texttt{scale} \tag{17.2.7}$$

where

$$\texttt{scale} = \texttt{atol} + |y|\,\texttt{rtol} \tag{17.2.8}$$

Here `atol` is the absolute error tolerance and `rtol` is the relative error tolerance. (Practical detail: In a code, you use $\max(|y_n|, |y_{n+1}|)$ for $|y|$ in the above formula in case one of them is close to zero.)

Our notation hides the fact that Δ is actually a vector of desired accuracies, Δ_i, one for each equation in the set of ODEs. In practice one takes some norm of the vector Δ. While taking the maximum component value is fine (i.e., rescaling the stepsize according to the needs of the "worst-offender" equation), we will use the usual Euclidean norm. Also, while `atol` and `rtol` could be different for each component of y, we will take them as constant. So define

$$\texttt{err} = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}\left(\frac{\Delta_i}{\texttt{scale}_i}\right)^2} \tag{17.2.9}$$

and accept the step if $\texttt{err} \le 1$, otherwise reject it.

What is the relation between the scaled error `err` and h? According to (17.2.4) – (17.2.5), Δ scales as h^5 and hence so does `err`. If we take a step h_1 and produce an error $\texttt{err}_1$, therefore, the step h_0 that *would have given* some other value $\texttt{err}_0$ is readily estimated as

$$h_0 = h_1 \left|\frac{\texttt{err}_0}{\texttt{err}_1}\right|^{1/5} \tag{17.2.10}$$

Let $\texttt{err}_0$ denote the desired error, which is 1 in an efficient integration. Then

equation (17.2.10) is used in two ways: If $\texttt{err}_1$ is larger than 1 in magnitude, the equation tells how much to decrease the stepsize *when we retry the present (failed) step*. If $\texttt{err}_1$ is smaller than 1, on the other hand, then the equation tells how much we can safely increase the stepsize *for the next step*. Local extrapolation means that we use the fifth-order value y_{n+1}, even though the error estimate actually applies to the fourth-order value y^*_{n+1}.

How is the quantity `err` related to some looser prescription like "get a solution good to one part in 10^6"? That can be a subtle question, and it depends on exactly what your application is. You may be dealing with a set of equations whose dependent variables differ enormously in magnitude. In that case, you probably want to use fractional errors, $\texttt{atol} = 0$, $\texttt{rtol} = \epsilon$, where ϵ is the number like 10^{-6} or whatever. On the other hand, you may have oscillatory functions that pass through zero but are bounded by some maximum values. In that case you probably want to set $\texttt{atol} = \texttt{rtol} = \epsilon$. This latter choice is the safest in general, and should usually be your first choice.

Here is a more technical point. The error criteria mentioned thus far are "local," in that they bound the error of each step individually. In some applications you may be unusually sensitive about a "global" accumulation of errors, from beginning to end of the integration and in the worst possible case where the errors all are presumed to add with the same sign. Then, the smaller the stepsize h, the *more steps* between your starting and ending values of x. In such a case you might want to set `scale` proportional to h, typically to something like

$$\texttt{scale} = \epsilon h \times \texttt{dydx[i]} \tag{17.2.11}$$

This enforces fractional accuracy ϵ not on the values of y but (much more stringently) on the *increments* to those values at each step. But now look back at (17.2.10). The exponent $1/5$ is no longer correct: When the stepsize is reduced from a too-large value, the new predicted value h_1 will fail to meet the desired accuracy when `scale` is also altered to this new h_1 value. Instead of $1/5$, we must scale by the exponent $1/4$ for things to work out.

Error control that tries to constrain the global error by setting the scale factor proportional to h is called "error per unit step," as opposed to the original "error per step" method. As a point of principle, controlling the global error by controlling the local error is very difficult. The global error at any point is the sum of the global error up to the start of the last step plus the local error of that step. This cumulative nature of the global error means it depends on things that cannot always be controlled, like stability properties of the differential equation. Accordingly, we recommend the straightforward "error per step" method in most cases. If you want to estimate the global error of your solution, you have to integrate again with a reduced tolerance and use the change in the solution as an estimate of the global error. This works *if* the stepsize controller produces errors roughly proportional to the tolerance, which is not always guaranteed.

Because our error estimates are not exact, but only accurate to the leading order in h, we are advised to put in a safety factor S that is a few percent smaller than unity. Equation (17.2.10) (with $\texttt{err}_0 = 1$ and the subscripts $1 \to n$ and $0 \to n+1$) is thus replaced by

$$h_{n+1} = S h_n \left(\frac{1}{\texttt{err}_n} \right)^{1/5} \tag{17.2.12}$$

Moreover, experience shows that it is not wise to let the stepsize increase or decrease too fast, and not to let the stepsize increase at all if the previous step was rejected. In `StepperDopr5`, the stepsize cannot increase by more than a factor of 10 nor decrease by more than a factor of 5 in a single step.

17.2.1 PI Stepsize Control

One situation in which the above stepsize controller has difficulty is when the stepsize is being limited by the stability properties of the integration method, rather than the accuracy of the individual steps. (We will see more about this in §17.5 on stiff differential equations.) The stepsize increases slowly as successive steps are accepted, until the method becomes unstable. The controller responds to the sudden increase in the error by cutting the stepsize drastically, and the cycle repeats itself. Similar problems can occur when the solution to the differential equation enters a region with drastically different behavior than the previous region. A long sequence of alternating accepted and rejected steps ensues. Since rejected steps are expensive, it is worth improving the stepsize control.

The most effective way to do this seems to be to use ideas from *control theory*. The integration routine and the differential equation play the role of the *process*, like a chemical plant manufacturing a product. The stepsize h is the input and the error estimate `err` is the output. (The numerical solution is also output, but it is not used for stepsize control.) The *controller* is the stepsize control algorithm. It tries to hold the error at the prescribed tolerance by varying the stepsize. Deriving an improved stepsize controller from control theory ideas is beyond our scope here, so we will introduce some basic concepts and then refer you to the literature for derivations and a fuller explanation [6-8].

The standard stepsize controller (17.2.12), when expressed in the language of control theory, is known as an *integrating controller*, with $\log h$ as the discrete control variable. This means that the control variable is obtained by "integrating" the control error signal. It is well known in control theory that more stable control can be achieved by adding an additional term *proportional* to the control error. This is called a PI controller, where the P stands for proportional feedback and the I for integral feedback. Instead of (17.2.12), the resulting algorithm takes the simple form

$$h_{n+1} = S h_n \mathtt{err}_n^{-\alpha} \mathtt{err}_{n-1}^{\beta} \tag{17.2.13}$$

Typically α and β should be scaled as $1/k$, where k is the exponent of h in `err` ($k = 5$ for a fifth-order method). Setting $\alpha = 1/k$, $\beta = 0$ recovers the classical controller (17.2.12). Nonzero β improves the stability but loses some efficiency for "easy" parts of the solution. A good compromise [6] is to set

$$\beta \approx 0.4/k, \qquad \alpha \approx 0.7/k = 1/k - 0.75\beta \tag{17.2.14}$$

17.2.2 Dense Output

Adaptive stepsize control means the algorithm marches along producing y values at x's that it chooses itself. What if you want output at values that you specify? The simplest option is just to integrate from one desired output point to the next. But if you specify a lot of output points, this is inefficient: The code has to take steps

based on where you want output, rather than the "natural" stepsizes it would like to choose. High-order methods like to take large steps for smooth solutions, so the problem is especially acute in this case.

The solution is to find an *interpolation* method that uses information produced during the integration and is of an order comparable to the order of the method so that full accuracy of the solution is preserved. This is called providing a *dense output* method.

For example, any method has available y and $dy/dx = f$ at the beginning and end of the step. These four quantities specify a cubic interpolating polynomial:

$$y(x_n+\theta h) = (1-\theta)y_n+\theta y_{n+1}+\theta(\theta-1)[(1-2\theta)(y_{n+1}-y_n)+(\theta-1)hf_n+\theta hf_{n+1}] \tag{17.2.15}$$

where $0 \le \theta \le 1$. Evaluating this polynomial at any θ in the interval gives a value of y that is third-order accurate, as you can verify by Taylor expansion in h. (Equation 17.2.15 is an example of *Hermite interpolation*, which uses both function and derivative values.)

We are interested, however, in integration methods with order higher than three, so higher-order dense output formulas are needed. The general approach for Runge-Kutta methods is to regard the b_i coefficients in (17.2.4) as polynomials in θ instead of constants. This defines a continuous solution,

$$y(x_n + \theta h) = y_n + b_1(\theta)k_1 + b_2(\theta)k_2 + b_3(\theta)k_3 + b_4(\theta)k_4 + b_5(\theta)k_5 + b_6(\theta)k_6 \tag{17.2.16}$$

and we require the polynomials $b_i(\theta)$ to approximate the true solution to the required order. Equation (17.2.15) is a special case of this.

The Dormand-Prince fifth-order method allows dense output of order four without any further function evaluations. This is usually sufficient: The number of steps to get to a typical point scales as $1/h$, so the global error at that point is typically $O(h^5)$ (fourth order). (Fifth-order dense output, needed, for example, for full accuracy in $y'(x_n + \theta h)$, turns out to need two extra function evaluations per step.) `StepperDopr5` contains a dense output option based on the formulas in [3] as simplified in [4].

Dense output simplifies problems where you don't know in advance how far to integrate. You want to locate the position x_c where some condition is satisfied. Examples include integrating the equations of stellar structure out from the center of the star until the pressure goes to zero at the surface, or the study of limit cycles when one integrates until the solution reaches the Poincaré section for the first time. Write the condition as finding the zero of some function:

$$g\,(x, y_i(x)) = 0 \tag{17.2.17}$$

Monitor g in the output routine. When g changes sign between two steps, use the dense output routine to supply function values to your favorite root-finding routine, such as bisection or Newton's method.

17.2.3 Implementation

Here follows the implementation of the fifth-order Dormand-Prince method.

stepperdopr5.h

```
template <class D>
struct StepperDopr5 : StepperBase {
```

Dormand-Prince fifth-order Runge-Kutta step with monitoring of local truncation error to ensure accuracy and adjust stepsize.

```
    typedef D Dtype;                          Make the type of derivs available to odeint.
    VecDoub k2,k3,k4,k5,k6;
    VecDoub rcont1,rcont2,rcont3,rcont4,rcont5;
    VecDoub dydxnew;
    StepperDopr5(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
        const Doub atoll, const Doub rtoll, bool dens);
    void step(const Doub htry,D &derivs);
    void dy(const Doub h,D &derivs);
    void prepare_dense(const Doub h,D &derivs);
    Doub dense_out(const Int i, const Doub x, const Doub h);
    Doub error();
    struct Controller {
        Doub hnext,errold;
        bool reject;
        Controller();
        bool success(const Doub err, Doub &h);
    };
    Controller con;
};
```

The constructor simply invokes the base class instructor and initializes variables:

stepperdopr5.h

```
template <class D>
StepperDopr5<D>::StepperDopr5(VecDoub_IO &yy,VecDoub_IO &dydxx,Doub &xx,
    const Doub atoll,const Doub rtoll,bool dens) :
    StepperBase(yy,dydxx,xx,atoll,rtoll,dens), k2(n),k3(n),k4(n),k5(n),k6(n),
    rcont1(n),rcont2(n),rcont3(n),rcont4(n),rcont5(n),dydxnew(n) {
```

Input to the constructor are the dependent variable `y[0..n-1]` and its derivative `dydx[0..n-1]` at the starting value of the independent variable `x`. Also input are the absolute and relative tolerances, `atol` and `rtol`, and the boolean `dense`, which is `true` if dense output is required.

```
    EPS=numeric_limits<Doub>::epsilon();
}
```

The `step` method is the actual stepper. It attempts a step, invokes the controller to decide whether to accept the step or try again with a smaller stepsize, and sets up the coefficients in case dense output is needed between x and $x + h$.

stepperdopr5.h

```
template <class D>
void StepperDopr5<D>::step(const Doub htry,D &derivs) {
```

Attempts a step with stepsize `htry`. On output, `y` and `x` are replaced by their new values, `hdid` is the stepsize that was actually accomplished, and `hnext` is the estimated next stepsize.

```
    Doub h=htry;                              Set stepsize to the initial trial value.
    for (;;) {
        dy(h,derivs);                         Take a step.
        Doub err=error();                     Evaluate accuracy.
        if (con.success(err,h)) break;        Step rejected. Try again with reduced h set
        if (abs(h) <= abs(x)*EPS)                 by controller.
            throw("stepsize underflow in StepperDopr5");
    }
    if (dense)                                Step succeeded. Compute coefficients for dense
        prepare_dense(h,derivs);                  output.
    dydx=dydxnew;                             Reuse last derivative evaluation for next step.
    y=yout;
    xold=x;                                   Used for dense output.
    x += (hdid=h);
    hnext=con.hnext;
}
```

The algorithm routine dy does the six steps plus the seventh FSAL step, and computes y_{n+1} and the error Δ.

stepperdopr5.h

```
template <class D>
void StepperDopr5<D>::dy(const Doub h,D &derivs) {
```

Given values for n variables y[0..n-1] and their derivatives dydx[0..n-1] known at x, use the fifth-order Dormand-Prince Runge-Kutta method to advance the solution over an interval h and store the incremented variables in yout[0..n-1]. Also store an estimate of the local truncation error in yerr using the embedded fourth-order method.

```
    static const Doub c2=0.2,c3=0.3,c4=0.8,c5=8.0/9.0,a21=0.2,a31=3.0/40.0,
    a32=9.0/40.0,a41=44.0/45.0,a42=-56.0/15.0,a43=32.0/9.0,a51=19372.0/6561.0,
    a52=-25360.0/2187.0,a53=64448.0/6561.0,a54=-212.0/729.0,a61=9017.0/3168.0,
    a62=-355.0/33.0,a63=46732.0/5247.0,a64=49.0/176.0,a65=-5103.0/18656.0,
    a71=35.0/384.0,a73=500.0/1113.0,a74=125.0/192.0,a75=-2187.0/6784.0,
    a76=11.0/84.0,e1=71.0/57600.0,e3=-71.0/16695.0,e4=71.0/1920.0,
    e5=-17253.0/339200.0,e6=22.0/525.0,e7=-1.0/40.0;
    VecDoub ytemp(n);
    Int i;
    for (i=0;i<n;i++)                          First step.
        ytemp[i]=y[i]+h*a21*dydx[i];
    derivs(x+c2*h,ytemp,k2);                   Second step.
    for (i=0;i<n;i++)
        ytemp[i]=y[i]+h*(a31*dydx[i]+a32*k2[i]);
    derivs(x+c3*h,ytemp,k3);                   Third step.
    for (i=0;i<n;i++)
        ytemp[i]=y[i]+h*(a41*dydx[i]+a42*k2[i]+a43*k3[i]);
    derivs(x+c4*h,ytemp,k4);                   Fourth step.
    for (i=0;i<n;i++)
        ytemp[i]=y[i]+h*(a51*dydx[i]+a52*k2[i]+a53*k3[i]+a54*k4[i]);
    derivs(x+c5*h,ytemp,k5);                   Fifth step.
    for (i=0;i<n;i++)
        ytemp[i]=y[i]+h*(a61*dydx[i]+a62*k2[i]+a63*k3[i]+a64*k4[i]+a65*k5[i]);
    Doub xph=x+h;
    derivs(xph,ytemp,k6);                      Sixth step.
    for (i=0;i<n;i++)                          Accumulate increments with proper weights.
        yout[i]=y[i]+h*(a71*dydx[i]+a73*k3[i]+a74*k4[i]+a75*k5[i]+a76*k6[i]);
    derivs(xph,yout,dydxnew);                  Will also be first evaluation for next step.
    for (i=0;i<n;i++) {
        Estimate error as difference between fourth- and fifth-order methods.
        yerr[i]=h*(e1*dydx[i]+e3*k3[i]+e4*k4[i]+e5*k5[i]+e6*k6[i]+e7*dydxnew[i]);
    }
}
```

The routine prepare_dense uses the coefficients of [4] to set up the dense output quantities. Our coding of the dense output is closely based on that of the Fortran code DOPRI5 of [5].

stepperdopr5.h

```
template <class D>
void StepperDopr5<D>::prepare_dense(const Doub h,D &derivs) {
```

Store coefficients of interpolating polynomial for dense output in rcont1...rcont5.

```
    VecDoub ytemp(n);
    static const Doub d1=-12715105075.0/11282082432.0,
    d3=87487479700.0/32700410799.0, d4=-10690763975.0/1880347072.0,
    d5=701980252875.0/199316789632.0, d6=-1453857185.0/822651844.0,
    d7=69997945.0/29380423.0;
    for (Int i=0;i<n;i++) {
        rcont1[i]=y[i];
        Doub ydiff=yout[i]-y[i];
        rcont2[i]=ydiff;
        Doub bspl=h*dydx[i]-ydiff;
        rcont3[i]=bspl;
        rcont4[i]=ydiff-h*dydxnew[i]-bspl;
```

```
            rcont5[i]=h*(d1*dydx[i]+d3*k3[i]+d4*k4[i]+d5*k5[i]+d6*k6[i]+
                d7*dydxnew[i]);
        }
    }
```

The next routine, `dense_out`, uses the coefficients stored by the previous routine to evaluate the solution at an arbitrary point.

stepperdopr5.h

```
template <class D>
Doub StepperDopr5<D>::dense_out(const Int i,const Doub x,const Doub h) {
```

Evaluate interpolating polynomial for `y[i]` at location `x`, where `xold` $\leq$ `x` $\leq$ `xold` $+$ `h`.

```
    Doub s=(x-xold)/h;
    Doub s1=1.0-s;
    return rcont1[i]+s*(rcont2[i]+s1*(rcont3[i]+s*(rcont4[i]+s1*rcont5[i])));
}
```

The `error` routine converts Δ into the scaled quantity `err`.

stepperdopr5.h

```
template <class D>
Doub StepperDopr5<D>::error() {
```

Use `yerr` to compute norm of scaled error estimate. A value less than one means the step was successful.

```
    Doub err=0.0,sk;
    for (Int i=0;i<n;i++) {
        sk=atol+rtol*MAX(abs(y[i]),abs(yout[i]));
        err += SQR(yerr[i]/sk);
    }
    return sqrt(err/n);
}
```

Finally, the `controller` tests whether `err` ≤ 1 and adjusts the stepsize. The default setting is `beta` $= 0$ (no PI control). Set `beta` to 0.04 or 0.08 to turn on PI control.

stepperdopr5.h

```
template <class D>
StepperDopr5<D>::Controller::Controller() : reject(false), errold(1.0e-4) {}
```

Step size controller for fifth-order Dormand-Prince method.

```
template <class D>
bool StepperDopr5<D>::Controller::success(const Doub err,Doub &h) {
```

Returns true if `err` ≤ 1, false otherwise. If step was successful, sets `hnext` to the estimated optimal stepsize for the next step. If the step failed, reduces `h` appropriately for another try.

```
    static const Doub beta=0.0,alpha=0.2-beta*0.75,safe=0.9,minscale=0.2,
        maxscale=10.0;
```

Set `beta` to a nonzero value for PI control. `beta` $= 0.04$–0.08 is a good default.

```
    Doub scale;
    if (err <= 1.0) {                    Step succeeded. Compute hnext.
        if (err == 0.0)
            scale=maxscale;
        else {                           PI control if beta ≠ 0.
            scale=safe*pow(err,-alpha)*pow(errold,beta);
            if (scale<minscale) scale=minscale;   Ensure minscale ≤ hnext/h ≤ maxscale.
            if (scale>maxscale) scale=maxscale;
        }
        if (reject)                      Don't let step increase if last one was re-
            hnext=h*MIN(scale,1.0);                                  jected.
        else
            hnext=h*scale;
        errold=MAX(err,1.0e-4);          Bookkeeping for next call.
        reject=false;
        return true;
    } else {                             Truncation error too large, reduce stepsize.
```

```
            scale=MAX(safe*pow(err,-alpha),minscale);
            h *= scale;
            reject=true;
            return false;
        }
    }
```

A warning: Don't be too greedy in specifying `atol` and `rtol`. The punishment for excessive greediness is interesting and worthy of Gilbert and Sullivan's *Mikado*: The routine can always achieve an apparent *zero* error by making the stepsize so small that quantities of order hy' add to quantities of order y as if they were zero. Then the routine chugs happily along taking infinitely many infinitesimal steps and never changing the dependent variables one iota. (On a supercomputer, you guard against this catastrophic loss of your time allocation by signaling on abnormally small stepsizes or on the dependent variable vector remaining unchanged from step to step. On a desktop computer, you guard against it by not taking too long a lunch hour while the program is running.)

17.2.4 Dopr853 — An Eighth-Order Method

Once you understand the above implementation of `StepperDopr5`, then you have the framework for essentially any Runge-Kutta method. For production work, we recommend that you use the following method, `StepperDopr853`. It is again a Dormand-Prince embedded method, this time of eighth order that uses 12 function evaluations. The original version used a sixth-order embedded method for error estimation. However, it turned out that the error estimation was not robust in certain circumstances because the last evaluation point happened not to be used in computing the error. Accordingly, Hairer, Nörsett, and Wanner [5] constructed both fifth-order and third-order embedded methods that use the last point. Then the error is estimated as

$$\texttt{err} = \texttt{err}_5 \frac{\texttt{err}_5}{\sqrt{(\texttt{err}_3)^2 + 0.01(\texttt{err}_5)^2}} \tag{17.2.18}$$

Most of the time, $\texttt{err}_5 \ll \texttt{err}_3$, so $\texttt{err} = O(h^8)$. If the estimation breaks down so that either $\texttt{err}_3$ gets small or $\texttt{err}_5$ gets large, then `err` will still give a reasonable basis for stepsize control. This method has worked well in practice and is the basis for the "853" in the name of the method.

For an eighth-order method we would like seventh-order dense output. It turns out this requires three more function evaluations. Our coding of the dense output follows closely the Fortran implementation of [5]. Since the code is somewhat lengthy, but basically similar to `StepperDopr5`, we give it as `StepperDopr853` in a Webnote [9].

CITED REFERENCES AND FURTHER READING:

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall).[1]

Dormand, J.R, and Prince, P.J. 1980, "A Family of Embedded Runge-Kutta Formulae," *Journal of Computational and Applied Mathematics*, vol. 6, pp. 19–26.[2]

Shampine, L.F., and Watts, H.A. 1977, "The Art of Writing a Runge-Kutta Code, Part I," in *Mathematical Software III*, J.R. Rice, ed. (New York: Academic Press), pp. 257–275; 1979, "The Art of Writing a Runge-Kutta Code. II," *Applied Mathematics and Computation*, vol. 5, pp. 93–121.

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall).

Dormand, J.R, and Prince, P.J. 1986, "Runge-Kutta Triples," *Computers and Mathematics with Applications*, vol. 12A, pp. 1007–1017.[3]

Shampine, L.F. 1986, "Some Practical Runge-Kutta Formulas," *Mathematics of Computation*, vol. 46, pp. 135–150.[4]

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at `http://www.unige.ch/~hairer/software.html`.[5]

Gustafsson, K. 1991, "Control Theoretic Techniques for Stepsize Selection in Explicit Runge-Kutta Methods," *ACM Transactions on Mathematical Software*, vol. 17, pp. 533-554.[6]

Hairer, E., Nørsett, S.P., and Wanner, G. 1996, *Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems*, 2nd ed. (New York: Springer), p. 28.[7]

Söderlind, G. 2003, "Digital Filters in Adaptive Time-stepping," *ACM Transactions on Mathematical Software*, vol. 29, pp. 1–26.[8]

Numerical Recipes Software 2007, "Routine Implementing an Eighth-order Runge-Kutta Method," *Numerical Recipes Webnote No. 20*, at `http://www.nr.com/webnotes?20` [9]

17.3 Richardson Extrapolation and the Bulirsch-Stoer Method

The techniques in this section are for differential equations containing smooth functions. With just three caveats, we believe that the Bulirsch-Stoer method, discussed here, is the best-known way to obtain high accuracy solutions to ordinary differential equations with minimal computational effort. The caveats are these:

- If you have a nonsmooth problem, for example, a differential equation whose right-hand side involves a function that is evaluated by table look-up and interpolation, go back to Runge-Kutta with an adaptive stepsize choice. That method does an excellent job of feeling its way through rocky or discontinuous terrain. It is also an excellent choice for a quick-and-dirty, low accuracy solution of a set of equations.
- The techniques in this section are not particularly good for differential equations that have singular points *inside* the interval of integration. A regular solution must tiptoe very carefully across such points. Runge-Kutta with adaptive stepsize can sometimes effect this; more generally, there are special techniques available for such problems, beyond our scope here but touched on in §18.6.
- There *may* be a few problems that are both very smooth and have right-hand sides that are very expensive to evaluate, for which predictor-corrector methods, discussed in §17.6, are the methods of choice.

The methods in this section involve three key ideas. The first is *Richardson's deferred approach to the limit*, which we already met in §4.3 on Romberg integration. The idea is to consider the final answer of a numerical calculation as itself being an analytic function (if a complicated one) of an adjustable parameter like the stepsize h. That analytic function can be probed by performing the calculation with various values of h, *none* of them being necessarily small enough to yield the accuracy that we desire. When we know enough about the function, we *fit* it to some analytic form

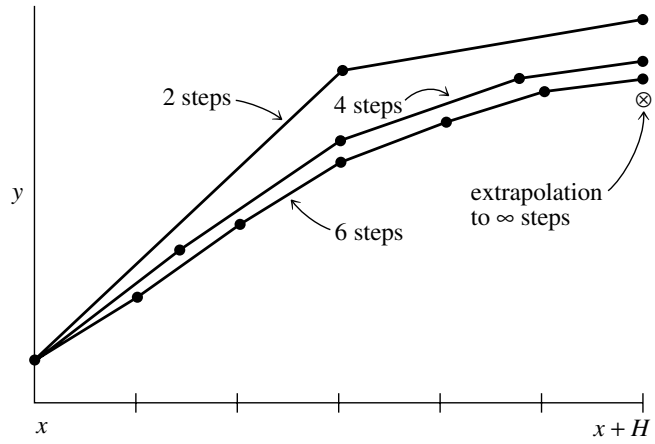


Figure 17.3.1. Richardson extrapolation as used in the Bulirsch-Stoer method. A large interval H is spanned by different sequences of finer and finer substeps. Their results are extrapolated to an answer that is supposed to correspond to infinitely fine substeps. In the Bulirsch-Stoer method, the integrations are done by the modified midpoint method, and the extrapolation technique is polynomial extrapolation.

and then *evaluate* it at that mythical and golden point $h = 0$ (see Figure 17.3.1). Richardson extrapolation is a method for turning straw into gold! (Lead into gold for alchemist readers.)

The second idea has to do with what kind of fitting function is used. Bulirsch and Stoer first recognized the strength of *rational function extrapolation* in Richardson-type applications. That strength is to break the shackles of the power series and its limited radius of convergence, out only to the distance of the first pole in the complex plane. Rational function fits can remain good approximations to analytic functions even after the various terms in powers of h all have comparable magnitudes. In other words, h can be so large as to make the whole notion of the "order" of the method meaningless — and the method can still work superbly. Nevertheless, more recent experience suggests that for smooth problems straightforward polynomial extrapolation is slightly more efficient than rational function extrapolation. (This may tell us more about the kinds of problems used for tests than about the methods themselves.) In any event, we will adopt polynomial extrapolation as our default. You might wish at this point to review §3.2, where polynomial function extrapolation was already discussed.

The third idea is to to use an integration method whose error function is strictly *even*, allowing the rational function or polynomial approximation to be in terms of the variable h^2 instead of just h. We will expand on this idea in the next subsection, on the modified midpoint method.

Put these ideas together and you have the *Bulirsch-Stoer method* [1]. A single Bulirsch-Stoer step takes us from x to $x + H$, where H is supposed to be quite a large — not at all infinitesimal — distance. That single step is a grand leap consisting of many (e.g., dozens to hundreds) substeps of the modified midpoint method, which are then extrapolated to zero stepsize.

17.3.1 Modified Midpoint Method

The *modified midpoint method* advances a vector of dependent variables $y(x)$ from a point x to a point $x + H$ by a sequence of n substeps each of size h,

$$h = H/n \tag{17.3.1}$$

In principle, one could use the modified midpoint method in its own right as an ODE integrator. In practice, the method finds its most important application as a part of the more powerful Bulirsch-Stoer technique,.

The number of right-hand side evaluations required by the modified midpoint method is $n + 1$. The formulas for the method are

$$\begin{aligned} z_0 &\equiv y(x) \\ z_1 &= z_0 + hf(x, z_0) \\ z_{m+1} &= z_{m-1} + 2hf(x + mh, z_m) \qquad \text{for} \quad m = 1, 2, \ldots, n-1 \\ y(x + H) \approx y_n &\equiv \tfrac{1}{2}[z_n + z_{n-1} + hf(x + H, z_n)] \end{aligned} \tag{17.3.2}$$

Here the z's are intermediate approximations that march along in steps of h, while y_n is the final approximation to $y(x + H)$. The method is basically a "centered difference" or "midpoint" method (compare equation 17.1.2), except at the first and last points. Those give the qualifier "modified."

The modified midpoint method is a second-order method, like (17.1.2), but with the advantage of requiring (asymptotically for large n) only one derivative evaluation per step h instead of the two required by second-order Runge-Kutta.

The usefulness of the modified midpoint method to the Bulirsch-Stoer technique (§17.3) derives from a "deep" result about equations (17.3.2), due to Gragg. It turns out that the error of (17.3.2), expressed as a power series in h, the stepsize, contains only *even* powers of h,

$$y_n - y(x + H) = \sum_{i=1}^{\infty} \alpha_i h^{2i} \tag{17.3.3}$$

where H is held constant but h changes by varying n in (17.3.1). The importance of this even power series is that, if we play our usual tricks of combining steps to knock out higher-order error terms, we can gain *two* orders at a time!

For example, suppose n is even, and let $y_{n/2}$ denote the result of applying (17.3.1) and (17.3.2) with half as many steps, $n \to n/2$. Then the estimate

$$y(x + H) \approx \frac{4y_n - y_{n/2}}{3} \tag{17.3.4}$$

is *fourth-order* accurate, the same as fourth-order Runge-Kutta, but requires only about 1.5 derivative evaluations per step h instead of Runge-Kutta's four evaluations. Don't be too anxious to implement (17.3.4), since we will soon do even better.

Now would be a good time to look back at the routine `qsimp` in §4.2, and especially to compare equation (4.2.4) with equation (17.3.4) above. You will see that the transition in Chapter 4 to the idea of Richardson extrapolation, as embodied in Romberg integration of §4.3, is exactly analogous to the transition in going from this section to the next one.

A routine that implements the modified midpoint method will be given as part of the implementation of `StepperBS`, in the `dy` member function.

17.3.2 The Bulirsch-Stoer Method

Consider attempting to cross the interval H using the modified midpoint method with increasing values of n, the number of substeps. Bulirsch and Stoer originally proposed the sequence

$$n = 2, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, \ldots, [n_j = 2n_{j-2}], \ldots \tag{17.3.5}$$

More recent work by Deuflhard [2,3] suggests that the sequence

$$n = 2, 4, 6, 8, 10, 12, 14, \ldots, [n_j = 2(j+1)], \ldots \tag{17.3.6}$$

is usually more efficient. For each step, we do not know in advance how far up this sequence we will go. After each successive n is tried, a polynomial extrapolation is attempted. That extrapolation gives both extrapolated values and error estimates. If the errors are not satisfactory, we go higher in n. If they are satisfactory, we go on to the next step and begin anew with $n = 2$.

Of course there must be some upper limit, beyond which we conclude that there is some obstacle in our path in the interval H, so that we must reduce H rather than just subdivide it more finely. Moreover, precision loss sets in if we choose too fine a subdivision. In the implementations below, the maximum number of n's to be tried is called `KMAXX`. We usually take this equal to 8; the eighth value of the sequence (17.3.6) is 16, so this is the maximum number of subdivisions of H that we allow.

We enforce error control, as in the Runge-Kutta method, by monitoring internal consistency and adapting the stepsize to match a prescribed bound on the local truncation error. Each new result from the sequence of modified midpoint integrations allows a tableau like that in equation (3.2.2) to be extended by one additional set of diagonals. Write the tableau as a lower triangular matrix:

$$\begin{array}{llll} T_{00} & & & \\ T_{10} & T_{11} & & \\ T_{20} & T_{21} & T_{22} & \\ \cdots & \cdots & \cdots & \cdots \end{array} \tag{17.3.7}$$

Here $T_{k0} = y_k$, where y_k is $y(x_n + H)$ computed with the stepsize $h_k = H/n_k$. Neville's algorithm, equation (3.2.3), with P replaced by T, $x_i = h_i^2$, and $x = 0$, can be written

$$T_{k,j+1} = T_{kj} + \frac{T_{kj} - T_{k-1,j}}{(n_k/n_{k-j})^2 - 1} \qquad j = 0, 1, \ldots, k-1 \tag{17.3.8}$$

Each new stepsize h_i starts a new row in the tableau, and then the polynomial extrapolation fills the rest of the row. Each new element in the tableau comes from the two closest elements in the previous column. Elements in the same column have the same order, and T_{kk}, the last element in each row, is the highest-order approximation with that stepsize. The difference between the last two elements in a row is taken as the (conservative) error estimate. How should we use this error estimate to adjust the stepsize? A good strategy was originally proposed by Deuflhard [2,3]. We will use a modified version [4], next described.

17.3.3 Stepsize Control Algorithm for Bulirsch-Stoer

The elements in the tableau are actually vectors corresponding to the vector y of dependent variables. Accordingly, define

$$\texttt{err}_k = \|T_{kk} - T_{k,k-1}\| \tag{17.3.9}$$

where the norm is the same scaled norm used in equation (17.2.9). Error control is enforced by requiring $\texttt{err}_k \leq 1$.

Now T_{kk} is of order $2k+2$ and $T_{k,k-1}$ is of order $2k$, which is therefore the order of $\texttt{err}_k$. In other words,

$$\texttt{err}_k \sim H^{2k+1} \tag{17.3.10}$$

Thus a simple estimate of a new stepsize H_k to obtain convergence in a fixed column k would be (cf. equation 17.2.12)

$$H_k = HS_1 \left(\frac{S_2}{\texttt{err}_k} \right)^{1/(2k+1)} \tag{17.3.11}$$

where S_1 and S_2 are safety factors smaller than one.

Which column k should we aim to achieve convergence in? Let's compare the work required for different k. Suppose A_k is the work to obtain row k of the extrapolation tableau. Assume the work is dominated by the cost of evaluating the functions defining the right-hand sides of the differential equations. For n_k subdivisions in H, the number of function evaluations can be found from the recurrence

$$\begin{aligned} A_0 &= n_0 + 1 \\ A_{k+1} &= A_k + n_{k+1} \end{aligned} \tag{17.3.12}$$

The work per unit step to get column k is therefore

$$W_k = \frac{A_k}{H_k} \tag{17.3.13}$$

The optimal column index is the one that minimizes W_k. The strategy is to set a target k for the next step, and then choose the stepsize from (17.3.11) to try to get convergence (i.e., $\texttt{err}_k \leq 1$) for that value of k on the next step.

In practice, you compute the extrapolation tableau (17.3.7) row by row, but only test for convergence within an *order window* between $k-1$ and $k+1$. The rationale for the order window is that if convergence appears to occur before column $k-1$, it is often spurious, resulting from some fortuitously small error estimate in the extrapolation. On the other hand, if you need to go beyond $k+1$ to obtain convergence, your local model of the convergence behavior is obviously not very good and you need to cut the stepsize and reestablish it.

Here are the steps:

- Test for convergence in column $k-1$: If $\texttt{err}_{k-1} \leq 1$, accept $T_{k-1,k-1}$. Set the new target as

$$k_{\text{new}} = \begin{cases} k-2 & \text{if } W_{k-2} < 0.8W_{k-1} \text{ (order decrease)} \\ k & \text{if } W_{k-1} < 0.9W_{k-2} \text{ (order increase)} \\ k-1 & \text{otherwise} \end{cases} \tag{17.3.14}$$

  Set the corresponding stepsize as

$$H_{\text{new}} = \begin{cases} H_{k_{\text{new}}} & \text{if } k_{\text{new}} = k-1 \text{ or } k-2 \\ H_{k-1}\dfrac{A_k}{A_{k-1}} & \text{if } k_{\text{new}} = k \end{cases} \tag{17.3.15}$$

  The idea behind the last formula is that you can't set $H_{\text{new}} = H_k$ because you're stopping the integration in row $k-1$ so you don't compute H_k. However, since k is supposedly optimal, $W_k \approx W_{k-1}$, which gives the last formula for H_{new}.

- If $\texttt{err}_{k-1} > 1$: Check if you can expect convergence by row $k+1$ by estimating what $\texttt{err}_{k+1}$ will be. Assuming one is in the asymptotic regime, one can show that

$$\texttt{err}_k \approx \left(\frac{n_0}{n_k}\right)^2 \texttt{err}_{k-1} \tag{17.3.16}$$

  and hence that $\texttt{err}_{k+1}$ will be greater than one if approximately

$$\texttt{err}_{k-1} > \left(\frac{n_k}{n_0}\right)^2 \left(\frac{n_{k+1}}{n_0}\right)^2 \tag{17.3.17}$$

  If this condition is satisfied, reject the step and restart with k_{new} and H_{new} chosen according to (17.3.14) and (17.3.15).

- If (17.3.17) is not satisfied, compute the next row of the tableau (i.e., for the target value of k) and see if convergence is attained for column k. Thus, if $\texttt{err}_k \le 1$, accept the step and continue with

$$k_{\text{new}} = \begin{cases} k-1 & \text{if } W_{k-1} < 0.8W_k \text{ (order decrease)} \\ k+1 & \text{if } W_k < 0.9W_{k-1} \text{ (order increase)} \\ k & \text{otherwise} \end{cases} \tag{17.3.18}$$

  Set the corresponding stepsize as

$$H_{\text{new}} = \begin{cases} H_{k_{\text{new}}} & \text{if } k_{\text{new}} = k \text{ or } k-1 \\ H_k \dfrac{A_{k+1}}{A_k} & \text{if } k_{\text{new}} = k+1 \end{cases} \tag{17.3.19}$$

- If $\texttt{err}_k > 1$, check if you can expect convergence by the next row. Analogously to (17.3.17), check if

$$\texttt{err}_k > \left(\frac{n_{k+1}}{n_0}\right)^2 \tag{17.3.20}$$

  If this condition is satisfied, reject the step and restart with k_{new} and H_{new} chosen according to (17.3.18) and (17.3.19).

- If (17.3.17) is not satisfied, compute row $k+1$ of the tableau. If $\texttt{err}_{k+1} \le 1$, accept the step. Set the new target with the following prescription:

$$\begin{aligned} &k_{\text{new}} = k \\ &\text{if } W_{k-1} < 0.8W_k \qquad k_{\text{new}} = k-1 \text{ (order decrease)} \\ &\text{if } W_{k+1} < 0.9W_{k_{\text{new}}} \qquad k_{\text{new}} = k+1 \text{ (order increase)} \end{aligned} \tag{17.3.21}$$

  The stepsize is set as in (17.3.19).

- If $\texttt{err}_{k+1} > 1$, reject the step. Restart with k_{new} and H_{new} chosen according to (17.3.18) and (17.3.19).

Two important refinements to this strategy are

- After a step is rejected, the order and stepsize are not allowed to increase.
- H_{new} computed from equation (16.4.5) is not allowed to change too rapidly in one step. It is restricted by

$$\frac{F}{S_4} \le \frac{H_{\text{new}}}{H} \le \frac{1}{F} \qquad F \equiv S_3^{1/(2k+1)} \tag{17.3.22}$$

  The default values of the parameters are $S_3 = 0.02$, $S_4 = 4$.

For the first step, the target k is estimated crudely from the requested precision, but the step is accepted if the error is small enough for any smaller k. For the last step, the stepsize is decreased to be the length of the remaining integration interval, so a similar increase in the order window is allowed.

17.3.4 Dense Output

The basic Bulirsch-Stoer step H is typically much larger than in Runge-Kutta methods because of the high orders invoked, so a dense output option is even more important. Our implementation once again is based closely on the coding in [4], which is based on [5].

A dense output algorithm turns out to be possible only for certain stepsize sequences, for example increasing by fours:

$$n = 2, 6, 10, 14, 18, 22, 26, 30, \ldots \tag{17.3.23}$$

The idea is to do Hermite interpolation using the function and derivative values at the beginning and end of the step. These are supplemented with values of the function and its derivatives at the midpoint obtained by extrapolation of values saved during the integration.

The error of the Hermite interpolation needs to be monitored. If it is too big, the step is rejected and the stepsize reduced appropriately. The error estimate of the interpolation is also used if necessary to limit the size of the next step after a successful step.

17.3.5 Implementation

The use of `StepperBS` is exactly the same as the use of the Runge-Kutta routines. For example, to solve the problem at the end of §17.2, everything is exactly the same except the line

```
Odeint<StepperDopr5<rhs_van> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
```

is replaced by

```
Odeint<StepperBS<rhs_van> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
```

The object `StepperBS` implements a Bulirsch-Stoer step. Some of its functions are declared `virtual` because the algorithm `StepperStoerm` will be implemented as a derived class from it in the next section, and these functions will be overridden. As with `StepperDopr5`, the class is templated on the functor class defining the right-hand side derivatives.

stepperbs.h

```
template <class D>
struct StepperBS : StepperBase {
```

Bulirsch-Stoer step with monitoring of local truncation error to ensure accuracy and adjust stepsize.

```
    typedef D Dtype;                    Make the type of derivs available to odeint.
    static const Int KMAXX=8,IMAXX=KMAXX+1;
```

`KMAXX` is the maximum number of rows used in the extrapolation.

```
    Int k_targ;                         Optimal row number for convergence.
    VecInt nseq;                        Stepsize sequence.
    VecInt cost;                        A_k.
    MatDoub table;                      Extrapolation tableau.
    VecDoub dydxnew;
    Int mu;                             Used for dense output.
    MatDoub coeff;                      Coefficients used in extrapolation tableau.
    VecDoub errfac;                     Used to compute dense interpolation error.
    MatDoub ysave;                      ysave and fsave store values and derivatives to be
    MatDoub fsave;                          used for dense output.
    VecInt ipoint;                      Keeps track of where values are stored in fsave.
    VecDoub dens;                       Stores quantities for dense interpolating polynomial.
    StepperBS(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx, const Doub atol,
```

```
        const Doub rtol, bool dens);
    void step(const Doub htry,D &derivs);
    virtual void dy(VecDoub_I &y, const Doub htot, const Int k, VecDoub_O &yend,
        Int &ipt, D &derivs);
    void polyextr(const Int k, MatDoub_IO &table, VecDoub_IO &last);
    virtual void prepare_dense(const Doub h,VecDoub_I &dydxnew, VecDoub_I &ysav,
        VecDoub_I &scale, const Int k, Doub &error);
    virtual Doub dense_out(const Int i,const Doub x,const Doub h);
    virtual void dense_interp(const Int n, VecDoub_IO &y, const Int imit);
};
```

Detailed implementations of the member functions are given in a Webnote [6].

CITED REFERENCES AND FURTHER READING:

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §7.2.14.[1]

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), §6.1.4 and §6.2.

Deuflhard, P. 1983, "Order and Stepsize Control in Extrapolation Methods," *Numerische Mathematik*, vol. 41, pp. 399–422.[2]

Deuflhard, P. 1985, "Recent Progress in Extrapolation Methods for Ordinary Differential Equations," *SIAM Review*, vol. 27, pp. 505–535.[3]

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at `http://www.unige.ch/~hairer/software.html`.[4]

Hairer, E., and Ostermann, A. 1990, "Dense Output for Extrapolation Methods," *Numerische Mathematik*, vol. 58, pp. 419–439.[5]

Numerical Recipes Software 2007, "StepperBS Implementations," *Numerical Recipes Webnote No. 21*, at `http://www.nr.com/webnotes?21` [6]

17.4 Second-Order Conservative Equations

Usually when you have a system of high-order differential equations to solve it is best to reformulate them as a system of first-order equations, as discussed in §17.0. There is a particular class of equations that occurs quite frequently in practice where you can gain about a factor of two in efficiency by differencing the equations directly. The equations are second-order systems where the derivative does not appear on the right-hand side:

$$y'' = f(x, y), \qquad y(x_0) = y_0, \qquad y'(x_0) = z_0 \tag{17.4.1}$$

As usual, y can denote a vector of values.

Stoermer's rule, dating back to 1907, has been a popular method for discretizing such systems. With $h = H/m$ we have

$$\begin{aligned} y_1 &= y_0 + h[z_0 + \tfrac{1}{2}hf(x_0, y_0)] \\ y_{k+1} - 2y_k + y_{k-1} &= h^2 f(x_0 + kh, y_k), \qquad k = 1, \ldots, m-1 \\ z_m &= (y_m - y_{m-1})/h + \tfrac{1}{2}hf(x_0 + H, y_m) \end{aligned} \tag{17.4.2}$$

Here z_m is $y'(x_0 + H)$. Henrici showed how to rewrite equations (17.4.2) to reduce roundoff error by using the quantities $\Delta_k \equiv y_{k+1} - y_k$. Start with

$$\begin{aligned} \Delta_0 &= h[z_0 + \tfrac{1}{2}hf(x_0, y_0)] \\ y_1 &= y_0 + \Delta_0 \end{aligned} \tag{17.4.3}$$

Then, for $k = 1, \ldots, m-1$, set

$$\begin{aligned} \Delta_k &= \Delta_{k-1} + h^2 f(x_0 + kh, y_k) \\ y_{k+1} &= y_k + \Delta_k \end{aligned} \tag{17.4.4}$$

Finally compute the derivative from

$$z_m = \Delta_{m-1}/h + \tfrac{1}{2} h f(x_0 + H, y_m) \tag{17.4.5}$$

Gragg again showed that the error series for equations (17.4.3) – (17.4.5) contains only even powers of h, and so the method is a logical candidate for extrapolation à la Bulirsch-Stoer.

Here is the `StepperStoerm` routine:

stepperstoerm.h

```
template <class D>
struct StepperStoerm : StepperBS<D> {
```

Stoermer's rule for integrating $y'' = f(x, y)$ for a system of equations.

```
    using StepperBS<D>::x; using StepperBS<D>::xold; using StepperBS<D>::y;
    using StepperBS<D>::dydx; using StepperBS<D>::dense; using StepperBS<D>::n;
    using StepperBS<D>::KMAXX; using StepperBS<D>::IMAXX; using StepperBS<D>::nseq;
    using StepperBS<D>::cost; using StepperBS<D>::mu; using StepperBS<D>::errfac;
    using StepperBS<D>::ysave; using StepperBS<D>::fsave;
    using StepperBS<D>::dens; using StepperBS<D>::neqn;
    MatDoub ysavep;
    StepperStoerm(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
        const Doub atol, const Doub rtol, bool dens);
    void dy(VecDoub_I &y, const Doub htot, const Int k, VecDoub_O &yend,
        Int &ipt,D &derivs);
    void prepare_dense(const Doub h,VecDoub_I &dydxnew, VecDoub_I &ysav,
        VecDoub_I &scale, const Int k, Doub &error);
    Doub dense_out(const Int i,const Doub x,const Doub h);
    void dense_interp(const Int n, VecDoub_IO &y, const Int imit);
};
```

Because the base class `StepperBS` is templated on the `derivs` class, the derived class `StepperStoerm` does not automatically inherit its member variables. This is the reason for the `using` declarations.

Note that in order to reuse the `StepperBS` code and make `StepperStoerm` a derived class, the arrays `y` and `dydx` are of length $2n$ for a system of n second-order equations. The values of y are stored in the first n elements of `y`, while the first derivatives are stored in the second n elements. The right-hand side f is stored in the first n elements of the array `dydx`, which therefore actually contains y''; the second n elements are unused.

The constructor has to redefine the costs A_k because there are half the number of function evaluations per step compared with the midpoint method:

stepperstoerm.h

```
template <class D>
StepperStoerm<D>::StepperStoerm(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
    const Doub atoll,const Doub rtoll, bool dens)
    : StepperBS<D>(yy,dydxx,xx,atoll,rtoll,dens),ysavep(IMAXX,n/2) {
```

Constructor. On input, `y[0..n-1]` contains y in its first `n`/2 elements and y' in its second `n`/2 elements, all evaluated at `x`. The vector `dydx[0..n-1]` contains the right-hand side function f (also evaluated at `x`) in its first `n`/2 elements. Its second `n`/2 elements are not referenced. Also input are the absolute and relative tolerances, `atol` and `rtol`, and the boolean `dense`, which is `true` if dense output is required.

```
    neqn=n/2;
    cost[0]=nseq[0]/2+1;
    for (Int k=0;k<KMAXX;k++)
        cost[k+1]=cost[k]+nseq[k+1]/2;
    for (Int i=0; i<2*IMAXX+1; i++) {
        Int ip7=i+7;
        Doub fac=1.5/ip7;
        errfac[i]=fac*fac*fac;
        Doub e = 0.5*sqrt(Doub(i+1)/ip7);
```

Number of equations.

Redefine cost: half as many function evaluations as Bulirsch-Stoer.

Coefficients for interpolation error are different too.

```
        for (Int j=0; j<=i; j++) {
            errfac[i] *= e/(j+1);
        }
    }
}
```

Here is the routine dy that implements Stoermer's rule:

stepperstoerm.h

```
template <class D>
void StepperStoerm<D>::dy(VecDoub_I &y, const Doub htot, const Int k,
    VecDoub_O &yend, Int &ipt, D &derivs) {
```

Stoermer step. Inputs are y, H, and k. The output is returned as `yend[0..2n-1]`. The counter `ipt` keeps track of saving the right-hand sides in the correct locations for dense output.

```
    VecDoub ytemp(n);
    Int nstep=nseq[k];
    Doub h=htot/nstep;                          Stepsize this trip.
    Doub h2=2.0*h;
    for (Int i=0;i<neqn;i++) {                  First step.
        ytemp[i]=y[i];
        Int ni=neqn+i;
        ytemp[ni]=y[ni]+h*dydx[i];
    }
    Doub xnew=x;
    Int nstp2=nstep/2;
    for (Int nn=1;nn<=nstp2;nn++) {             General step.
        if (dense && nn == (nstp2+1)/2) {
            for (Int i=0;i<neqn;i++) {
                ysavep[k][i]=ytemp[neqn+i];
                ysave[k][i]=ytemp[i]+h*ytemp[neqn+i];
            }
        }
        for (Int i=0;i<neqn;i++)
            ytemp[i] += h2*ytemp[neqn+i];
        xnew += h2;
        derivs(xnew,ytemp,yend);                Store derivatives temporarily in yend.
        if (dense && abs(nn-(nstp2+1)/2) < k+1) {
            ipt++;
            for (Int i=0;i<neqn;i++)
                fsave[ipt][i]=yend[i];
        }
        if (nn != nstp2) {
            for (Int i=0;i<neqn;i++)
                ytemp[neqn+i] += h2*yend[i];
        }
    }
    for (Int i=0;i<neqn;i++) {                  Last step.
        Int ni=neqn+i;
        yend[ni]=ytemp[ni]+h*yend[i];
        yend[i]=ytemp[i];
    }
}
```

The dense output routines are in a Webnote [1].

CITED REFERENCES AND FURTHER READING:

Deuflhard, P. 1985, "Recent Progress in Extrapolation Methods for Ordinary Differential Equations," *SIAM Review*, vol. 27, pp. 505–535.

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at `http://www.unige.ch/~hairer/software.html`.

Numerical Recipes Software 2007, "Dense Output for Stoermer's Rule," *Numerical Recipes Webnote No. 22*, at http://www.nr.com/webnotes?22 [1]

17.5 Stiff Sets of Equations

As soon as one deals with more than one first-order differential equation, the possibility of a *stiff* set of equations arises. Stiffness typically occurs in a problem where there are two or more very different scales of the independent variable on which the dependent variables are changing. For example, consider the following set of equations [1]:

$$\begin{aligned} u' &= 998u + 1998v \\ v' &= -999u - 1999v \end{aligned} \tag{17.5.1}$$

with boundary conditions

$$u(0) = 1 \qquad v(0) = 0 \tag{17.5.2}$$

By means of the transformation

$$u = 2y - z \qquad v = -y + z \tag{17.5.3}$$

we find the solution

$$\begin{aligned} u &= 2e^{-x} - e^{-1000x} \\ v &= -e^{-x} + e^{-1000x} \end{aligned} \tag{17.5.4}$$

If we integrated the system (17.5.1) with any of the methods given so far in this chapter, the presence of the e^{-1000x} term would require a stepsize $h \ll 1/1000$ for the method to be stable (the reason for this is explained below). This is so even though the e^{-1000x} term is completely negligible in determining the values of u and v as soon as one is away from the origin (see Figure 17.5.1).

This is the generic disease of stiff equations: We are required to follow the variation in the solution on the shortest length scale to maintain the stability of the integration, even though accuracy requirements allow a much larger stepsize.

To see how we might cure this problem, consider the single equation

$$y' = -cy \tag{17.5.5}$$

where $c > 0$ is a constant. The explicit (or *forward*) Euler scheme for integrating this equation with stepsize h is

$$y_{n+1} = y_n + hy'_n = (1 - ch)y_n \tag{17.5.6}$$

The method is called explicit because the new value y_{n+1} is given explicitly in terms of the old value y_n. Clearly the method is unstable if $h > 2/c$, for then $|y_n| \to \infty$ as $n \to \infty$ even though the solution of (17.5.5) is bounded.

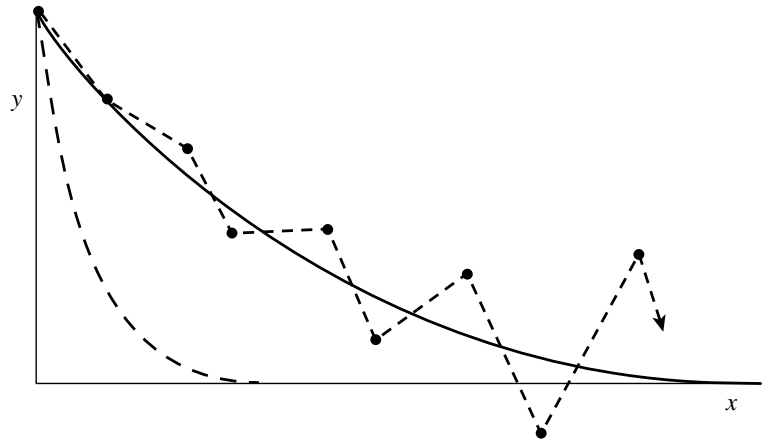


Figure 17.5.1. Example of an instability encountered in integrating a stiff equation (schematic). Here it is supposed that the equation has two solutions, shown as solid and dashed lines. Although the initial conditions are such as to give the solid solution, the stability of the integration (shown as the unstable dotted sequence of segments) is determined by the more rapidly varying dashed solution, even after that solution has effectively died away to zero. Implicit integration methods are the cure.

The simplest cure is to resort to *implicit* differencing, where the right-hand side is evaluated at the *new* y location. In this case, we get the *backward Euler* scheme:

$$y_{n+1} = y_n + hy'_{n+1} \tag{17.5.7}$$

or

$$y_{n+1} = \frac{y_n}{1 + ch} \tag{17.5.8}$$

The method is absolutely stable: Even as $h \to \infty$, $y_{n+1} \to 0$, which is in fact the correct solution of the differential equation. If we think of x as representing time, then the implicit method converges to the true equilibrium solution (i.e., the solution at late times) for large stepsizes. This nice feature of implicit methods holds only for linear systems, but even in the general case implicit methods give better stability. Of course, we give up *accuracy* in following the evolution toward equilibrium if we use large stepsizes, but we maintain *stability*.

These considerations can easily be generalized to sets of linear equations with constant coefficients:

$$\mathbf{y}' = -\mathbf{C} \cdot \mathbf{y} \tag{17.5.9}$$

Consider first the usual case where the matrix $\mathbf{C}$ can be diagonalized by a similarity transformation (cf. eqn. 11.0.11)

$$\mathbf{T} \cdot \mathbf{C} \cdot \mathbf{T}^{-1} = \text{diag}(\lambda_0 \dots \lambda_{N-1}) \tag{17.5.10}$$

where λ_i are the eigenvalues of $\mathbf{C}$. If we define the vector $\mathbf{z}(x)$ by $\mathbf{z} = \mathbf{T}^{-1} \cdot \mathbf{y}(x)$, then equation (17.5.9) becomes

$$\mathbf{z}' = -\text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{z} \tag{17.5.11}$$

This is a set of N independent equations for the components of $\mathbf{z}$ with solution

$$\mathbf{z} = \text{diag}(e^{-\lambda_0 x} \dots e^{-\lambda_{N-1} x}) \cdot \mathbf{z}_0 \tag{17.5.12}$$

Thus the solution of the original equation is

$$\mathbf{y} = \mathbf{T} \cdot \text{diag}(e^{-\lambda_0 x} \dots e^{-\lambda_{N-1} x}) \cdot \mathbf{T}^{-1} \cdot \mathbf{y}_0 \tag{17.5.13}$$

We will be interested in the *stable* solutions, that is, those that decay as $x \to \infty$. (This notion can be made more rigorous by considering *Liapunov stability*, the idea that if $\mathbf{y}_0$ is small then $\mathbf{y}$ is small for all $x > 0$.) From equation (17.5.13) we see that the criterion for stable solutions is

$$\text{Re}\,\lambda_i > 0 \qquad i = 0, \dots, N-1 \tag{17.5.14}$$

What if the matrix $\mathbf{C}$ in equation (17.5.9) cannot be diagonalized? Then it can always be transformed to so-called Jordan canonical form, which is the "closest" it can come to being made diagonal. Using this form, one can show that criterion (17.5.14) is still the stability criterion [2].

Now consider solving equation (17.5.9) by explicit differencing as in equation (17.5.6):

$$\mathbf{y}_{n+1} = (\mathbf{1} - \mathbf{C}h) \cdot \mathbf{y}_n \tag{17.5.15}$$

and so

$$\mathbf{y}_n = (\mathbf{1} - \mathbf{C}h)^n \cdot \mathbf{y}_0 \tag{17.5.16}$$

If $\mathbf{C}$ can be diagonalized, it has a complete set of eigenvectors $\{\boldsymbol{\xi}_i\}$ that can be used as a basis to expand $\mathbf{y}_0$:

$$\mathbf{y}_0 = \sum_{i=0}^{N-1} \alpha_i \boldsymbol{\xi}_i \tag{17.5.17}$$

Substituting this expansion in equation (17.5.16) gives

$$\mathbf{y}_n = \sum_{i=0}^{N-1} \alpha_i (1 - h\lambda_i)^n \boldsymbol{\xi}_i \tag{17.5.18}$$

If the original differential equation is stable, we require the difference scheme to be stable in that it must have bounded solutions, that is, $\mathbf{y}_n$ must be bounded as $n \to \infty$. From equation (17.5.18) we see that stability of the difference scheme requires

$$|1 - h\lambda_i| < 1 \qquad i = 0, \dots, N-1 \tag{17.5.19}$$

If the λ_i are all real, then since they are positive for the differential equation to be stable, the stability criterion for the difference scheme is

$$h < \frac{2}{\lambda_{\max}} \tag{17.5.20}$$

where $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{C}$.

As usual, if $\mathbf{C}$ cannot be diagonalized and so does not have a complete set of eigenvectors, then by working with the Jordan canonical form one can show the same result.

Consider now implicit differencing, which gives

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{y}'_{n+1} \tag{17.5.21}$$

or

$$\mathbf{y}_{n+1} = (\mathbf{1} + \mathbf{C}h)^{-1} \cdot \mathbf{y}_n \tag{17.5.22}$$

Criterion (17.5.19) becomes

$$|1 + h\lambda_i|^{-1} < 1 \qquad i = 0, \ldots, N-1 \tag{17.5.23}$$

which is satisfied for all h — the method is stable for all stepsizes. The penalty we pay for this stability is that we are required to invert a matrix at each step.

Not all equations are linear with constant coefficients, unfortunately! For the system

$$\mathbf{y}' = \mathbf{f}(\mathbf{y}) \tag{17.5.24}$$

implicit differencing gives

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(\mathbf{y}_{n+1}) \tag{17.5.25}$$

In general this is some nasty set of nonlinear equations that has to be solved iteratively at each step. Suppose we try linearizing the equations, as in Newton's method:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\left[\mathbf{f}(\mathbf{y}_n) + \left.\frac{\partial \mathbf{f}}{\partial \mathbf{y}}\right|_{\mathbf{y}_n} \cdot (\mathbf{y}_{n+1} - \mathbf{y}_n)\right] \tag{17.5.26}$$

Here $\partial \mathbf{f}/\partial \mathbf{y}$ is the matrix of the partial derivatives of the right-hand side (the Jacobian matrix). Rearrange equation (17.5.26) into the form

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\left[\mathbf{1} - h\frac{\partial \mathbf{f}}{\partial \mathbf{y}}\right]^{-1} \cdot \mathbf{f}(\mathbf{y}_n) \tag{17.5.27}$$

If h is not too big, only one iteration of Newton's method may be accurate enough to solve equation (17.5.25) using equation (17.5.27). In other words, at each step we have to invert the matrix

$$\mathbf{1} - h\frac{\partial \mathbf{f}}{\partial \mathbf{y}} \tag{17.5.28}$$

to find $\mathbf{y}_{n+1}$. Solving implicit methods by linearization is called a "semi-implicit" method, so equation (17.5.27) is the *semi-implicit Euler method*. It is not guaranteed to be stable, but it usually is, because the behavior is locally similar to the case of a constant matrix $\mathbf{C}$ described above.

So far we have dealt only with implicit methods that are first-order accurate. While these are very robust, most problems will benefit from higher-order methods. There are three important classes of higher-order methods for stiff systems:

- Generalizations of the Runge-Kutta method. These consist of implicit methods, where nonlinear equations are solved by Newton iteration at each step, and semi-implicit methods that solve linear equations analogous to (17.5.27). These semi-implicit methods are often called Rosenbrock methods. The first good implementation of these ideas was by Kaps and Rentrop, and so these methods are also called Kaps-Rentrop methods.
- Generalizations of the Bulirsch-Stoer method, which extrapolate a semi-implicit sequence of integrations to zero stepsize.
- Predictor-corrector methods, most of which are descendants of Gear's backward differentiation method.

We shall give implementations of Rosenbrock and extrapolation methods. An example of a good implicit Runge-Kutta code in Fortran is RADAU [3], while several stiff predictor-corrector–type Fortran codes (LSODE, DEBDF, VODE and MEBDF) are available from Netlib [5].

Here is an important point: *It is absolutely crucial to scale your variables properly when integrating stiff problems with automatic stepsize adjustment.* As in our nonstiff routines, you will be asked to supply absolute and relative tolerances `atol` and `rtol`. In stiff problems, there are often strongly decreasing pieces of the solution that you are not particularly interested in following once they are small. Thus you should almost never integrate with a pure relative error criterion by setting `atol` $= 0$. A good default choice is `atol` $=$ `rtol`, or sometimes a few orders of magnitude smaller.

One final warning: Solving stiff problems can sometimes lead to catastrophic precision loss. Double precision is often a requirement, not an option.

17.5.1 Rosenbrock Methods

These methods have the advantage of being relatively simple to understand and implement. For moderate accuracies (tolerances of order 10^{-4}–10^{-5}) and moderate-sized systems ($N \lesssim 10$), they are competitive with the more complicated algorithms. For more stringent parameters, Rosenbrock methods remain reliable; they merely become less efficient than competitors like the semi-implicit extrapolation method (see below).

A Rosenbrock method seeks a solution of the form

$$\mathbf{y}(x_0 + h) = \mathbf{y}_0 + \sum_{i=1}^{s} b_i \mathbf{k}_i \tag{17.5.29}$$

where the corrections $\mathbf{k}_i$ are found by solving s linear equations that generalize the structure in (17.5.27):

$$(\mathbf{1} - \gamma h \mathbf{f}') \cdot \mathbf{k}_i = h\mathbf{f}\left(\mathbf{y}_0 + \sum_{j=1}^{i-1} \alpha_{ij}\mathbf{k}_j\right) + h\mathbf{f}' \cdot \sum_{j=1}^{i-1} \gamma_{ij}\mathbf{k}_j, \qquad i = 1, \ldots, s \tag{17.5.30}$$

Here we denote the Jacobian matrix by $\mathbf{f}'$. The coefficients γ, b_i, α_{ij}, and γ_{ij} are fixed constants independent of the problem. If $\gamma = \gamma_{ij} = 0$, this is simply a Runge-Kutta scheme. Equations (17.5.30) can be solved successively for $\mathbf{k}_1, \mathbf{k}_2, \ldots$.

To minimize the matrix-vector multiplications on the right-hand side of (17.5.30), rewrite the equations in terms of quantities

$$\mathbf{g}_i = \sum_{j=1}^{i-1} \gamma_{ij}\mathbf{k}_j + \gamma\mathbf{k}_i \tag{17.5.31}$$

The equations then take the form (for four stages as an example)

$$\begin{aligned}
(\mathbf{1}/\gamma h - \mathbf{f}') \cdot \mathbf{g}_1 &= \mathbf{f}(\mathbf{y}_0) \\
(\mathbf{1}/\gamma h - \mathbf{f}') \cdot \mathbf{g}_2 &= \mathbf{f}(\mathbf{y}_0 + a_{21}\mathbf{g}_1) + c_{21}\mathbf{g}_1/h \\
(\mathbf{1}/\gamma h - \mathbf{f}') \cdot \mathbf{g}_3 &= \mathbf{f}(\mathbf{y}_0 + a_{31}\mathbf{g}_1 + a_{32}\mathbf{g}_2) + (c_{31}\mathbf{g}_1 + c_{32}\mathbf{g}_2)/h \\
(\mathbf{1}/\gamma h - \mathbf{f}') \cdot \mathbf{g}_4 &= \mathbf{f}(\mathbf{y}_0 + a_{41}\mathbf{g}_1 + a_{42}\mathbf{g}_2 + a_{43}\mathbf{g}_3) + (c_{41}\mathbf{g}_1 + c_{42}\mathbf{g}_2 + c_{43}\mathbf{g}_3)/h
\end{aligned} \tag{17.5.32}$$

Here a_{ij} and c_{ij} can be expressed in terms of α_{ij} and γ_{ij}.

Note that systems where the right-hand side $\mathbf{f}(\mathbf{y}, x)$ depends explicitly on x can be handled by adding x to the list of dependent variables so that the system to be solved is

$$\begin{pmatrix} \mathbf{y} \\ x \end{pmatrix}' = \begin{pmatrix} \mathbf{f} \\ 1 \end{pmatrix} \tag{17.5.33}$$

In the routine given below, we have explicitly carried out this replacement for you, so the routines can handle right-hand sides of the form $\mathbf{f}(\mathbf{y}, x)$ without any special effort on your part.

Crucial to the success of a stiff integration scheme is an automatic stepsize adjustment algorithm. Kaps and Rentrop [6] discovered an *embedded* or Runge-Kutta-Fehlberg method as described in §17.2: Two estimates of the form (17.5.29) are computed, the "real" one $\mathbf{y}$ and a lower-order estimate $\hat{\mathbf{y}}$ with different coefficients $\hat{b}_i, i = 1, \ldots, \hat{s}$, where $\hat{s} < s$ but the $\mathbf{k}_i$ are the same. The difference between $\mathbf{y}$ and $\hat{\mathbf{y}}$ leads to an estimate of the local truncation error, which can then be used for stepsize control. Kaps and Rentrop showed that the smallest value of s for which embedding is possible is $s = 4$, $\hat{s} = 3$, leading to a fourth-order method. By a suitable choice of parameters, only three function evaluations are needed for the four stages in each step.

In recent years, Kaps-Rentrop has lost favor to so-called *stiffly stable* methods, an implementation of which we give here as the routine `StepperRoss` (Rosenbrock Stiffly Stable), based on the Fortran routine RODAS [3]. It is also a fourth-order method with a third-order embedded method for stepsize control. Despite having six stages with six function evaluations, the enhanced stability makes it significantly more efficient than the Kaps-Rentrop method. Moreover, it has a simple dense output feature.

As with the earlier stepper routines in this chapter, you have to provide a functor for `derivs`, the right-hand side routine. In the structure you now must also supply a function called `jacobian` that returns $\mathbf{f}'$ and $\partial\mathbf{f}/\partial x$ as functions of x and $\mathbf{y}$. If x does not occur explicitly on the right-hand side, then `dfdx` will be zero. Usually the Jacobian matrix will be available to you by analytic differentiation of the right-hand side $\mathbf{f}$. If not, your routine will have to compute it by numerical differencing with appropriate increments $\Delta\mathbf{y}$. We will give an example of a complete derivative and jacobian structure at the end of this subsection.

The class `StepperRoss` uses a set of constants, which are provided by deriving the class from a class `Ross_constants`. This latter class is listed in a Webnote [4]. Here is the declaration of `StepperRoss`:

stepperross.h

```
template <class D>
struct StepperRoss : StepperBase, Ross_constants {
```

Fourth-order stiffly stable Rosenbrock step for integrating stiff ODEs, with monitoring of local truncation error to adjust stepsize.

```
    typedef D Dtype;
    MatDoub dfdy;
    VecDoub dfdx;
    VecDoub k1,k2,k3,k4,k5,k6;
    VecDoub cont1,cont2,cont3,cont4;
    MatDoub a;
    StepperRoss(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx, const Doub atoll,
        const Doub rtoll, bool dens);
    void step(const Doub htry,D &derivs);
    void dy(const Doub h,D &derivs);
    void prepare_dense(const Doub h,VecDoub_I &dydxnew);
    Doub dense_out(const Int i, const Doub x, const Doub h);
    Doub error();
    struct Controller {
        Doub hnext;
        bool reject;
        bool first_step;
        Doub errold;
        Doub hold;
        Controller();
        bool success(Doub err, Doub &h);
    };
    Controller con;
};
```

Make the type of `derivs` available to `odeint`.

$\mathbf{f}'$

$\partial\mathbf{f}/\partial x$

`first_step`, `errold`, and `hold` are used by the predictive controller.

The implementation will seem very familiar if you've looked at `StepperDopr5`, the explicit Runge-Kutta routine. Note that in the algorithm routine `dy` of `StepperRoss`, the linear equations (17.5.32) are solved by first computing the LU decomposition of the matrix

$\mathbf{1}/\gamma h - \mathbf{f}'$ using the routine LUdcmp. Then the six $\mathbf{g}_i$ are found by backsubstitution of the six different right-hand sides using the routine solve in LUdcmp. Thus each step of the integration requires one call to jacobian and six calls to derivs (one call outside dy and five calls inside). The evaluation of the Jacobian matrix is roughly equivalent to N evaluations of the right-hand side $\mathbf{f}$ (although it can often be less than this, especially if commonality of code can be exploited). Thus this scheme involves about $N + 6$ function evaluations per step. Note that if N is large and the Jacobian matrix is sparse, you should replace the LU decomposition by a suitable sparse matrix procedure.

stepperross.h

```
template <class D>
StepperRoss<D>::StepperRoss(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
    const Doub atoll,const Doub rtoll, bool dens) :
    StepperBase(yy,dydxx,xx,atoll,rtoll,dens),dfdy(n,n),dfdx(n),k1(n),k2(n),
    k3(n),k4(n),k5(n),k6(n),cont1(n),cont2(n),cont3(n),cont4(n),a(n,n) {
```

Input to the constructor are the dependent variable y[0..n-1] and its derivative dydx[0..n-1] at the starting value of the independent variable x. Also input are the absolute and relative tolerances, atol and rtol, and the boolean dense, which is true if dense output is required.

```
    EPS=numeric_limits<Doub>::epsilon();
}
template <class D>
void StepperRoss<D>::step(const Doub htry,D &derivs) {
```

Attempts a step with stepsize htry. On output, y and x are replaced by their new values, hdid is the stepsize that was actually accomplished, and hnext is the estimated next stepsize.

```
    VecDoub dydxnew(n);
    Doub h=htry;                          Set stepsize to the initial trial value.
    derivs.jacobian(x,y,dfdx,dfdy);       Compute the Jacobian and ∂f/∂x.
    for (;;) {
        dy(h,derivs);                     Take a step.
        Doub err=error();                 Evaluate accuracy.
        if (con.success(err,h)) break;    Step rejected. Try again with reduced h set
        if (abs(h) <= abs(x)*EPS)             by controller.
            throw("stepsize underflow in StepperRoss");
    }
    derivs(x+h,yout,dydxnew);             Step succeeded.
    if (dense)                            Compute coefficients for dense output.
        prepare_dense(h,dydxnew);
    dydx=dydxnew;                         Reuse last derivative evaluation for next step.
    y=yout;
    xold=x;                               Used for dense output.
    x += (hdid=h);
    hnext=con.hnext;
}
template<class D>
void StepperRoss<D>::dy(const Doub h,D &derivs) {
```

Given values for n variables y[0..n-1] and their derivatives dydx[0..n-1] known at x, use the fourth-order stiffly stable Rosenbrock method to advance the solution over an interval h and store the incremented variables in yout[0..n-1]. Also store an estimate of the local truncation error in yerr using the embedded third-order method.

```
    VecDoub ytemp(n),dydxnew(n);
    Int i;
    for (i=0;i<n;i++) {                   Set up the matrix 1/γh − f'.
        for (Int j=0;j<n;j++) a[i][j] = -dfdy[i][j];
        a[i][i] += 1.0/(gam*h);
    }
    LUdcmp alu(a);                        LU decomposition of the matrix.
    for (i=0;i<n;i++)                     Set up right-hand side for g1.
        ytemp[i]=dydx[i]+h*d1*dfdx[i];
     alu.solve(ytemp,k1);                 Solve for g1.
    for (i=0;i<n;i++)                     Compute intermediate values of y.
        ytemp[i]=y[i]+a21*k1[i];
    derivs(x+c2*h,ytemp,dydxnew);         Compute dydx at the intermediate values.
    for (i=0;i<n;i++)                     Set up right-hand side for g2.
        ytemp[i]=dydxnew[i]+h*d2*dfdx[i]+c21*k1[i]/h;
```

```
    alu.solve(ytemp,k2);                    Solve for g_2.
    for (i=0;i<n;i++)                       Compute intermediate values of y.
        ytemp[i]=y[i]+a31*k1[i]+a32*k2[i];
    derivs(x+c3*h,ytemp,dydxnew);           Compute dydx at the intermediate values.
    for (i=0;i<n;i++)                       Set up right-hand side for g_3.
        ytemp[i]=dydxnew[i]+h*d3*dfdx[i]+(c31*k1[i]+c32*k2[i])/h;
    alu.solve(ytemp,k3);                    Solve for g_3.
    for (i=0;i<n;i++)                       Compute intermediate values of y.
        ytemp[i]=y[i]+a41*k1[i]+a42*k2[i]+a43*k3[i];
    derivs(x+c4*h,ytemp,dydxnew);           Compute dydx at the intermediate values.
    for (i=0;i<n;i++)                       Set up right-hand side for g_4.
        ytemp[i]=dydxnew[i]+h*d4*dfdx[i]+(c41*k1[i]+c42*k2[i]+c43*k3[i])/h;
    alu.solve(ytemp,k4);                    Solve for g_4.
    for (i=0;i<n;i++)                       Compute intermediate values of y.
        ytemp[i]=y[i]+a51*k1[i]+a52*k2[i]+a53*k3[i]+a54*k4[i];
    Doub xph=x+h;
    derivs(xph,ytemp,dydxnew);              Compute dydx at the intermediate values.
    for (i=0;i<n;i++)                       Set up right-hand side for g_5.
        k6[i]=dydxnew[i]+(c51*k1[i]+c52*k2[i]+c53*k3[i]+c54*k4[i])/h;
    alu.solve(k6,k5);                       Solve for g_5.
    for (i=0;i<n;i++)                       Compute the embedded solution.
        ytemp[i] += k5[i];
    derivs(xph,ytemp,dydxnew);              Last derivative evaluation.
    for (i=0;i<n;i++)                       Compute the solution and the error.
        k6[i]=dydxnew[i]+(c61*k1[i]+c62*k2[i]+c63*k3[i]+c64*k4[i]+c65*k5[i])/h;
    alu.solve(k6,yerr);
    for (i=0;i<n;i++)
        yout[i]=ytemp[i]+yerr[i];
}
template <class D>
void StepperRoss<D>::prepare_dense(const Doub h,VecDoub_I &dydxnew) {
```

Store coefficients of interpolating polynomial for dense output in `cont1...cont4`.

```
    for (Int i=0;i<n;i++) {
        cont1[i]=y[i];
        cont2[i]=yout[i];
        cont3[i]=d21*k1[i]+d22*k2[i]+d23*k3[i]+d24*k4[i]+d25*k5[i];
        cont4[i]=d31*k1[i]+d32*k2[i]+d33*k3[i]+d34*k4[i]+d35*k5[i];
    }
}
template <class D>
Doub StepperRoss<D>::dense_out(const Int i,const Doub x,const Doub h) {
```

Evaluate interpolating polynomial for `y[i]` at location `x`, where `xold` $\leq$ `x` $\leq$ `xold` $+$ `h`.

```
    Doub s=(x-xold)/h;
    Doub s1=1.0-s;
    return cont1[i]*s1+s*(cont2[i]+s1*(cont3[i]+s*cont4[i]));
}
template <class D>
Doub StepperRoss<D>::error() {
```

Use `yerr` to compute norm of scaled error estimate. A value less than one means the step was successful.

```
    Doub err=0.0,sk;
    for (Int i=0;i<n;i++) {
        sk=atol+rtol*MAX(abs(y[i]),abs(yout[i]));
        err += SQR(yerr[i]/sk);
    }
    return sqrt(err/n);
}
```

Stepsize control depends on the fact that

$$\begin{aligned} \mathbf{y}_{\text{exact}} &= \mathbf{y} + O(h^5) \\ \mathbf{y}_{\text{exact}} &= \hat{\mathbf{y}} + O(h^4) \end{aligned} \tag{17.5.34}$$

Thus

$$|\mathbf{y} - \hat{\mathbf{y}}| = O(h^4) \tag{17.5.35}$$

Referring back to the steps leading from equation (17.2.4) to equation (17.2.12), we see that the new stepsize should be chosen as in equation (17.2.12) but with the exponent 1/5 replaced by 1/4. Also, experience shows that it is wise to prevent too large a stepsize change in one step, otherwise we will probably have to undo the large change in the next step. We adopt 0.2 and 6 as the maximum allowed decrease and increase of h in one step.

Methods for integrating stiff equations do not suffer from the stability limitations that led to the PI controller of §17.2.1. However, stiff problems often need a rapid decrease in stepsize even when the previous step is successful. Also, sometimes the effective order of the method can be lower than the simple Taylor series prediction. Gustafsson [7] has proposed a *predictive controller* that does a good job of dealing with these problems. The resulting formula is

$$h_{n+1} = S h_n \left(\frac{1}{\mathrm{err}_n}\right)^{1/4} \frac{h_n}{h_{n-1}} \left(\frac{\mathrm{err}_{n-1}}{\mathrm{err}_n}\right)^{1/4} \tag{17.5.36}$$

It is used only when a step is accepted.

stepperross.h

```
template <class D>
StepperRoss<D>::Controller::Controller() : reject(false), first_step(true) {}
```

Step size controller for fourth-order Rosenbrock method.

```
template <class D>
bool StepperRoss<D>::Controller::success(Doub err, Doub &h) {
```

Returns true if err ≤ 1, false otherwise. If step was successful, sets hnext to the estimated optimal stepsize for the next step. If the step failed, reduces h appropriately for another try.

```
    static const Doub safe=0.9,fac1=5.0,fac2=1.0/6.0;
    Doub fac=MAX(fac2,MIN(fac1,pow(err,0.25)/safe));
    Doub hnew=h/fac;                    Ensure 1/fac1 ≤ hnew/h ≤ 1/fac2.
    if (err <= 1.0) {                   Step succeeded.
        if (!first_step) {              Predictive control.
            Doub facpred=(hold/h)*pow(err*err/errold,0.25)/safe;
            facpred=MAX(fac2,MIN(fac1,facpred));
            fac=MAX(fac,facpred);
            hnew=h/fac;
        }
        first_step=false;
        hold=h;
        errold=MAX(0.01,err);
        if (reject)                     Don't let step increase if last one was rejected.
            hnew=(h >= 0.0 ? MIN(hnew,h) : MAX(hnew,h));
        hnext=hnew;
        reject=false;
        return true;
    } else {                            Truncation error too large, reduce stepsize.
        h=hnew;
        reject=true;
        return false;
    }
}
```

As an example of how `StepperRoss` is used, one can solve the system

$$\begin{aligned} y_0' &= -.013y_0 - 1000y_0y_2 \\ y_1' &= -2500y_1y_2 \\ y_2' &= -.013y_0 - 1000y_0y_2 - 2500y_1y_2 \end{aligned} \tag{17.5.37}$$

with initial conditions

$$y_0(0) = 1, \qquad y_1(0) = 1, \qquad y_2(0) = 0 \tag{17.5.38}$$

(This is test problem D4 in [8].) We integrate the system up to $x = 50$ with an initial stepsize of $h = 2.9 \times 10^{-4}$ using `Odeint`. We set `atol` $=$ `rtol` $= 10^{-5}$. The right-hand side routine for this problem is given below. Even though the ratio of largest to smallest decay constants for this problem is around 10^6, `StepperRoss` succeeds in integrating this set in only 11 steps with 67 function evaluations. By contrast, the explicit Runge-Kutta routine `StepperDopr5` requires almost 60,000 steps and over 400,000 function evaluations!

```
struct rhs {
    void operator() (const Doub x, VecDoub_I &y, VecDoub_O &dydx) {
        dydx[0] = -0.013*y[0]-1000.0*y[0]*y[2];
        dydx[1] = -2500.0*y[1]*y[2];
        dydx[2] = -0.013*y[0]-1000.0*y[0]*y[2]-2500.0*y[1]*y[2];
    }
    void jacobian(const Doub x, VecDoub_I &y, VecDoub_O &dfdx,
        MatDoub_O &dfdy) {
        Int n=y.size();
        for (Int i=0;i<n;i++) dfdx[i]=0.0;
        dfdy[0][0] = -0.013-1000.0*y[2];
        dfdy[0][1] = 0.0;
        dfdy[0][2] = -1000.0*y[0];
        dfdy[1][0] = 0.0;
        dfdy[1][1] = -2500.0*y[2];
        dfdy[1][2] = -2500.0*y[1];
        dfdy[2][0] = -0.013-1000.0*y[2];
        dfdy[2][1] = -2500.0*y[2];
        dfdy[2][2] = -1000.0*y[0]-2500.0*y[1];
    }
};
```

17.5.2 Semi-Implicit Extrapolation Method

The Bulirsch-Stoer method, which discretizes the differential equation using the modified midpoint rule, does not work for stiff problems. For many years, successful extrapolation-type routines for stiff equations were based on an algorithm of Bader and Deuflhard [9]. This algorithm uses a semi-implicit version of the midpoint method that has an even error series.

Not long afterward, however, Deuflhard [10] investigated a semi-implicit version of the Euler method, equation (17.5.27). This does not have an even error series. Nevertheless, it turns out that for high precision, using this method as the basis of an extrapolation scheme is even more efficient than using the semi-implicit midpoint rule. (Some theoretical insight into this behavior is provided in §VI.5 of [3].) Since `StepperRoss` is generally satisfactory for low precision, this method is a good companion. We give it as `StepperSie` ("Semi-Implicit Euler").

The basic equation of the method is equation (17.5.27) rewritten in the form

$$\left[\mathbf{1}/h - \frac{\partial \mathbf{f}}{\partial \mathbf{y}}\right] \cdot (\mathbf{y}_{n+1} - \mathbf{y}_n) = \mathbf{f}(\mathbf{y}_n) \tag{17.5.39}$$

A sequence of stepsizes $h_i = H/n_i$ is used with this equation to advance the solution a distance H. The linear equations are solved with LU decomposition. Polynomial extrapolation is used as in the original Bulirsch-Stoer method, except that in equation (17.3.8) the ratio of stepsizes is not squared because the error series is not even.

Instead of making the replacement (17.5.33) in the above formula, it turns out to be slightly better to add a single simplified Newton iteration of the fully implicit Euler step (17.5.25):

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(x_{n+1}, \mathbf{y}_{n+1}) \longrightarrow \left[\mathbf{1} - h\frac{\partial \mathbf{f}}{\partial \mathbf{y}}\right] \cdot (\mathbf{y}_{n+1} - \mathbf{y}_n) = h\mathbf{f}(x_{n+1}, \mathbf{y}_n) \tag{17.5.40}$$

This costs an extra function evaluation but avoids the computation of $\partial \mathbf{f}/\partial x$. In the code, we leave $\partial \mathbf{f}/\partial x$ as an argument of the `jacobian` function for compatibility with `StepperRoss`, but it is not used.

Another difference from `StepperRoss` is that the Jacobian does not have to be exact. Its main role is to ensure stability, not accuracy. Accordingly, the code has a test to see when the Jacobian needs to be recomputed.

Differences from `StepperBS` include

- The default stepsize sequence is

$$n = 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64, \ldots, [n_j = 2n_{j-2}], \ldots \tag{17.5.41}$$

- The work per unit step now includes the cost of Jacobian evaluations as well as function evaluations. We count one Jacobian evaluation as equivalent to five function evaluations by default, but it could be as large as N, the number of equations. The work per unit step also includes the cost of the LU decomposition and the backsubstitutions, each set by default to the cost of a function evaluation.
- Several checks for instability are included. If the estimated error $\texttt{err}_k$ starts increasing with k during a step, the step is restarted with the stepsize reduced by a factor of two. Similarly, a stability test is made for $k = 0, 1$ during the Euler step and the step is rejected if the test is failed. You could add a test for failure of the LU decomposition and similarly reduce the stepsize if that happened.

The routine, which is based on the Fortran routine SEULEX [3], next follows.

17.5.3 Implementation of Semi-Implicit Extrapolation Method

The routine `StepperSie` is an excellent routine for all stiff problems, competitive with the best Gear-type routines. `StepperRoss` is often better in execution time for moderate N and $\epsilon \lesssim 10^{-5}$. The detailed implementation is listed in a Webnote [11].

CITED REFERENCES AND FURTHER READING:

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall).[1]

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at `http://www.unige.ch/~hairer/software.html`.[2]

Hairer, E., Nørsett, S.P., and Wanner, G. 1996, *Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems*, 2nd ed. (New York: Springer).[3]

Numerical Recipes Software 2007, "Constants for Stiffly Stable Rosenbrock Method," *Numerical Recipes Webnote No. 23*, at `http://www.nr.com/webnotes?23` [4]

Netlib: `http://www.netlib.org/`.[5]

Kaps, P., and Rentrop, P. 1979, "Generalized Runge-Kutta Methods of Order Four with Stepsize Control for Stiff Ordinary Differential Equations," *Numerische Mathematik*, vol. 33, pp. 55–68.[6]

Gustafsson, K. 1994, "Control Theoretic Techniques for Stepsize Selection in Implicit Runge-Kutta Methods," *ACM Transactions on Mathematical Software*, vol. 20, pp. 496-517.[7]

Enright, W.H., and Pryce, J.D. 1987, "Two FORTRAN Packages for Assessing Initial Value Methods," *ACM Transactions on Mathematical Software*, vol. 13, pp. 1–27.[8]

Bader, G., and Deuflhard, P. 1983, "A Semi-Implicit Mid-Point Rule for Stiff Systems of Ordinary Differential Equations," *Numerische Mathematik*, vol. 41, pp. 373–398.[9]

Deuflhard, P. 1985, "Recent Progress in Extrapolation Methods for Ordinary Differential Equations," *SIAM Review*, vol. 27, pp. 505–535.[10]

Numerical Recipes Software 2007, "StepperSie Implementation," *Numerical Recipes Webnote No. 24*, at `http://www.nr.com/webnotes?24` [11]

Deuflhard, P. 1983, "Order and Stepsize Control in Extrapolation Methods," *Numerische Mathematik*, vol. 41, pp. 399–422.

Enright, W.H., Hull, T.E., and Lindberg, B. 1975, "Comparing Numerical Methods for Stiff Systems of ODE's," *BIT*, vol. 15, pp. 10–48.

Wanner, G. 1988, in *Numerical Analysis 1987*, Pitman Research Notes in Mathematics, vol. 170, D.F. Griffiths and G.A. Watson, eds. (Harlow, Essex, UK: Longman Scientific and Technical).

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer).

17.6 Multistep, Multivalue, and Predictor-Corrector Methods

The terms "multistep" and "multivalue" describe two different ways of implementing essentially the same integration technique for ODEs. Predictor-corrector is a particular subcategory of these methods — in fact, the most widely used. Accordingly, the name predictor-corrector is often loosely used to denote all these methods.

We suspect that predictor-corrector integrators have had their day, and that they are no longer the method of choice for most problems in ODEs. For high-precision applications, or applications where evaluations of the right-hand sides are expensive, Bulirsch-Stoer dominates. For convenience, or for moderate precision, adaptive-stepsize Runge-Kutta dominates. Predictor-corrector methods have been, we think, squeezed out in the middle. There is possibly only one exceptional case: high-precision solution of very smooth equations with very complicated right-hand sides, as we will describe later.

Nevertheless, these methods have had a long historical run. Textbooks are full of information on them, and there are a lot of standard ODE programs around that are based on predictor-corrector methods. Many capable researchers have a lot of experience with predictor-corrector routines, and they see no reason to make a precipitous change of habit. It is not a bad idea for you to be familiar with the principles involved, and even with the sorts of bookkeeping details that are the bane of these methods. Otherwise, there will be a big surprise in store when you first have to fix a problem in a predictor-corrector routine.

Let us first consider the multistep approach. Think about how integrating an ODE is different from finding the integral of a function: For a function, the integrand has a known dependence on the independent variable x and can be evaluated at will. For an ODE, the "integrand" is the right-hand side, which depends both on x and on the dependent variables y. Thus, to advance the solution of $y' = f(x, y)$ from x_n to x, we have

$$y(x) = y_n + \int_{x_n}^{x} f(x', y)\, dx' \tag{17.6.1}$$

In a single-step method like Runge-Kutta or Bulirsch-Stoer, the value y_{n+1} at x_{n+1} depends only on y_n. In a multistep method, we approximate $f(x, y)$ by a polynomial passing through *several* previous points $x_n, x_{n-1}, \ldots$ and possibly also through x_{n+1}. The result of evaluating the integral (17.6.1) at $x = x_{n+1}$ is then of the form

$$y_{n+1} = y_n + h(\beta_0 y'_{n+1} + \beta_1 y'_n + \beta_2 y'_{n-1} + \beta_3 y'_{n-2} + \cdots) \tag{17.6.2}$$

where y'_n denotes $f(x_n, y_n)$, and so on. If $\beta_0 = 0$, the method is explicit; otherwise it is implicit. The order of the method depends on how many previous steps we use to get each new value of y.

Consider how we might solve an implicit formula of the form (17.6.2) for y_{n+1}. Two methods suggest themselves: *functional iteration* and *Newton's method*. In functional iteration, we take some initial guess for y_{n+1}, insert it into the right-hand side of (17.6.2) to get an updated value of y_{n+1}, insert this updated value back into the right-hand side, and continue iterating. But how are we to get an initial guess for y_{n+1}? Easy! Just use some *explicit* formula of the same form as (17.6.2). This is called the *predictor step*. In the predictor step we are essentially *extrapolating* the polynomial fit to the derivative from the previous points to the new point x_{n+1} and then doing the integral (17.6.1) in a Simpson-like manner from x_n to x_{n+1}. The subsequent Simpson-like integration, using the prediction step's value of y_{n+1} to *interpolate* the derivative, is called the *corrector step*. The difference between the predicted and corrected function values supplies information on the local truncation error that can be used to control accuracy and to adjust stepsize.

If one corrector step is good, aren't many better? Why not use each corrector as an improved predictor and iterate to convergence on each step? Answer: Even if you had a *perfect* predictor, the step would still be accurate only to the finite order of the corrector. This incurable error term is on the same order as that which your iteration is supposed to cure, so you are at best changing only the coefficient in front of the error term by a fractional amount. So dubious an improvement is certainly not worth the effort. Your extra effort would be better spent in taking a smaller stepsize.

As described so far, you might think it desirable or necessary to predict several intervals ahead at each step, then to use all these intervals, with various weights, in a Simpson-like corrector step. That is not a good idea. Extrapolation is the least stable part of the procedure, and it is desirable to minimize its effect. Therefore, the integration steps of a predictor-corrector method are overlapping, each one involving several stepsize intervals h, but extending just one such interval farther than the previous ones. Only that one extended interval is extrapolated by each predictor step.

The most popular predictor-corrector methods are probably the Adams-Bashforth-Moulton schemes, which have good stability properties. The Adams-Bashforth part is the predictor. For example, the third-order case is

$$\text{predictor:} \qquad y_{n+1} = y_n + \frac{h}{12}(23y'_n - 16y'_{n-1} + 5y'_{n-2}) + O(h^4) \tag{17.6.3}$$

Here information at the current point x_n, together with the two previous points x_{n-1} and x_{n-2} (assumed equally spaced), is used to predict the value y_{n+1} at the next point, x_{n+1}. The Adams-Moulton part is the corrector. The third-order case is

$$\text{corrector:} \qquad y_{n+1} = y_n + \frac{h}{12}(5y'_{n+1} + 8y'_n - y'_{n-1}) + O(h^4) \tag{17.6.4}$$

Without the trial value of y_{n+1} from the predictor step to insert on the right-hand side, the corrector would be a nasty implicit equation for y_{n+1}. (Despite the names, these formulas are actually all due to Adams.)

There are actually three separate processes occurring in a predictor-corrector method: the predictor step, which we call P; the evaluation of the derivative y'_{n+1} from the latest value of y, which we call E; and the corrector step, which we call C.

In this notation, iterating m times with the corrector (a practice we inveighed against earlier) would be written P(EC)^m. One also has the choice of finishing with a C or an E step. The lore is that a final E is superior, so the strategy usually recommended is PECE.

Notice that a PC method with a fixed number of iterations (say, one) is an explicit method. When we fix the number of iterations in advance, the final value of y_{n+1} can be written as some complicated function of known quantities. Thus fixed iteration PC methods lose the strong stability properties of implicit methods and *should only be used for nonstiff problems*.

For stiff problems we *must* use an implicit method if we want to avoid having tiny stepsizes. (Not all implicit methods are good for stiff problems, but fortunately some good ones such as the Gear formulas are known.) We then appear to have two choices for solving the implicit equations: functional iteration to convergence, or Newton iteration. However, it turns out that for stiff problems functional iteration will not even converge unless we use tiny stepsizes, no matter how close our prediction is! Thus Newton iteration is usually an essential part of a multistep stiff solver. For convergence, Newton's method doesn't particularly care what the stepsize is, as long as the prediction is accurate enough.

Multistep methods, as we have described them so far, suffer from two serious difficulties when one tries to implement them:

- Since the formulas require results from equally spaced steps, adjusting the stepsize is difficult.
- Starting and stopping present problems. For starting, we need the initial values plus several previous steps to prime the pump. Stopping is a problem because equal steps are unlikely to land directly on the desired termination point.

Older implementations of PC methods have various cumbersome ways of dealing with these problems. For example, they might use Runge-Kutta to start and stop. Changing the stepsize requires considerable bookkeeping to do some kind of interpolation procedure. Fortunately, both these drawbacks disappear with the multivalue approach.

For multivalue methods (also called Nordsieck methods), the basic data available to the integrator are the first few terms of the Taylor series expansion of the solution at the current point x_n. The aim is to advance the solution and obtain the expansion coefficients at the next point x_{n+1}. This is in contrast to multistep methods, where the data are the values of the solution at $x_n, x_{n-1}, \ldots$. We'll illustrate the idea by considering a four-value method, for which the basic data are

$$\mathbf{y}_n \equiv \begin{pmatrix} y_n \\ hy'_n \\ (h^2/2)y''_n \\ (h^3/6)y'''_n \end{pmatrix} \tag{17.6.5}$$

It is also conventional to scale the derivatives with the powers of $h = x_{n+1} - x_n$ as shown. Note that here we use the vector notation $\mathbf{y}$ to denote the solution and its first few derivatives at a point, not the fact that we are solving a system of equations with many components y.

In terms of the data in (17.6.5), we can approximate the value of the solution y

at some point x:

$$y(x) = y_n + (x - x_n)y'_n + \frac{(x - x_n)^2}{2}y''_n + \frac{(x - x_n)^3}{6}y'''_n \tag{17.6.6}$$

Set $x = x_{n+1}$ in equation (17.6.6) to get an approximation to y_{n+1}. Differentiate equation (17.6.6) and set $x = x_{n+1}$ to get an approximation to y'_{n+1}, and similarly for y''_{n+1} and y'''_{n+1}. Call the resulting approximation $\widetilde{\mathbf{y}}_{n+1}$, where the tilde is a reminder that all we have done so far is a polynomial extrapolation of the solution and its derivatives; we have not yet used the differential equation. You can easily verify that

$$\widetilde{\mathbf{y}}_{n+1} = \mathbf{B} \cdot \mathbf{y}_n \tag{17.6.7}$$

where the matrix $\mathbf{B}$ is

$$\mathbf{B} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{17.6.8}$$

We now write the actual approximation to $\mathbf{y}_{n+1}$ that we will use by adding a correction to $\widetilde{\mathbf{y}}_{n+1}$:

$$\mathbf{y}_{n+1} = \widetilde{\mathbf{y}}_{n+1} + \alpha \mathbf{r} \tag{17.6.9}$$

Here $\mathbf{r}$ will be a fixed vector of numbers, in the same way that $\mathbf{B}$ is a fixed matrix. We fix α by requiring that the differential equation

$$y'_{n+1} = f(x_{n+1}, y_{n+1}) \tag{17.6.10}$$

be satisfied. The second of the equations in (17.6.9) is

$$hy'_{n+1} = h\widetilde{y}'_{n+1} + \alpha r_1 \tag{17.6.11}$$

and this will be consistent with (17.6.10) provided

$$r_1 = 1, \qquad \alpha = hf(x_{n+1}, y_{n+1}) - h\widetilde{y}'_{n+1} \tag{17.6.12}$$

The values of r_0, r_2, and r_3 are free for the inventor of a given four-value method to choose. Different choices give different orders of method (i.e., through what order in h the final expression 17.6.9 actually approximates the solution) and different stability properties.

An interesting result, not obvious from our presentation, is that multivalue and multistep methods are entirely equivalent. In other words, the value y_{n+1} given by a multivalue method with given $\mathbf{B}$ and $\mathbf{r}$ is exactly the same value given by some multistep method with given β's in equation (17.6.2). For example, it turns out that the Adams-Bashforth formula (17.6.3) corresponds to a four-value method with $r_0 = 0$, $r_2 = 3/4$, and $r_3 = 1/6$. The method is explicit because $r_0 = 0$. The Adams-Moulton method (17.6.4) corresponds to the implicit four-value method with $r_0 = 5/12$, $r_2 = 3/4$, and $r_3 = 1/6$. Implicit multivalue methods are solved the same way as implicit multistep methods: either by a predictor-corrector approach using an explicit method for the predictor, or by Newton iteration for stiff systems.

Why go to all the trouble of introducing a whole new method that turns out to be equivalent to a method you already knew? The reason is that multivalue methods allow an easy solution to the two difficulties we mentioned above in actually implementing multistep methods.

Consider first the question of stepsize adjustment. To change stepsize from h to h' at some point x_n, simply multiply the components of $\mathbf{y}_n$ in (17.6.5) by the appropriate powers of h'/h, and you are ready to continue to $x_n + h'$.

Multivalue methods also allow a relatively easy change in the *order* of the method: Simply change $\mathbf{r}$. The usual strategy for this is first to determine the new stepsize with the current order from the error estimate. Then check what stepsize would be predicted using an order one greater and one smaller than the current order. Choose the order that allows you to take the biggest next step. Being able to change order also allows an easy solution to the starting problem: Simply start with a first-order method and let the order automatically increase to the appropriate level.

For moderate accuracy requirements, the most efficient choice is almost always a Runge-Kutta routine like `StepperDopr853`. For high accuracy, `StepperBS` is both robust and efficient. For very smooth functions, a variable-order PC method can invoke very high orders. If the right-hand side of the equation is relatively complicated, so that the expense of evaluating it outweighs the bookkeeping expense, then the best PC packages can outperform Bulirsch-Stoer on such problems. As you can imagine, however, such a variable-stepsize, variable-order method is not trivial to program. If you suspect that your problem is suitable for this treatment, we recommend the use of a packaged PC routine. For further details consult [1-3].

Our prediction is that, as extrapolation methods like Bulirsch-Stoer continue to gain sophistication, they will eventually beat out PC methods in *all* applications. We are willing, however, to be corrected.

CITED REFERENCES AND FURTHER READING:

Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.[1]

Shampine, L.F., and Gordon, M.K. 1975, *Computer Solution of Ordinary Differential Equations. The Initial Value Problem.* (San Francisco: W.H Freeman).[2]

Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer).[3]

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 5.

Kahaner, D., Moler, C., and Nash, S. 1989, *Numerical Methods and Software* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 8.

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 7.

17.7 Stochastic Simulation of Chemical Reaction Networks

We are so used to thinking of chemical (or nuclear) reaction networks as implying sets of continuous differential equations, that it takes an effort to remember

their underlying discrete, atomic, nature. To give an example, we have all learned to translate a set of reactions like

$$\begin{aligned} A + X &\xrightarrow{k_0} 2X \\ X + Y &\xrightarrow{k_1} 2Y \\ Y &\xrightarrow{k_2} B \end{aligned} \tag{17.7.1}$$

into a set of differential equations (*rate equations*) governing the concentrations of each species,

$$\begin{aligned} \frac{d[A]}{dt} &= -k_0[A][X] \equiv -a_0 \\ \frac{d[X]}{dt} &= k_0[A][X] - k_1[X][Y] \equiv a_0 - a_1 \\ \frac{d[Y]}{dt} &= k_1[X][Y] - k_2[Y] \equiv a_1 - a_2 \\ \frac{d[B]}{dt} &= k_2[Y] \equiv a_2 \end{aligned} \tag{17.7.2}$$

where a_0, a_1, and a_2 are respectively the rates of the three reactions in equation (17.7.1).

Increasingly in biological applications, however, one is faced with situations where the actual numbers of reacting molecules is so small that discreteness effects and fluctuations become important. In such cases, one needs to replace continuum concentrations like $[X]$ and $[Y]$ with actual numbers of molecular species. The equations (17.7.2) now become meaningless. What we need to do is directly simulate the discrete reactions in (17.7.1), assigning a sequence of stochastically generated times, and corresponding discrete changes in species numbers, to the occurrences of each reaction. This task is known as *stochastic simulation*, from the original work of Gillespie [1]. Stochastic simulation is a remarkably simple, and elegant, technique. Like many powerful tools, it can be both used and misused, as we will discuss.

Before we get to the details, it is useful to formalize some aspects of the structure of equations (17.7.1) and (17.7.2). In general, we have M reactions occurring among N species. Each reaction $j = 0, \ldots, M-1$, has an instantaneous rate, denoted a_j. In the discrete case, $1/a_j$ is the mean time until the next occurrence of reaction j, if no other reaction happens first. An important point is that each rate a_j depends only on the numbers of those species on the left-hand side of reaction j, its *reactants*. Define a *reactant matrix* λ_{ij} by

$$\lambda_{ij} = \begin{cases} 1 & \text{if species } S_i \text{ is an input to reaction } j \\ 0 & \text{otherwise} \end{cases} \tag{17.7.3}$$

On the output side (*products*), each set of reactions j is characterized by a *state change matrix* ν_{ij} whose i, j component is the net change in the number of species S_i due to one occurrence of reaction j. (The jth column of this matrix is often called the *state change vector* for reaction j.) In terms of these quantities, the conventional

rate equations, like (17.7.2), can be written in general as

$$\frac{d[S_i]}{dt} = \sum_{j=0}^{M-1} \nu_{ij}\, a_j(\{[S_k]\}), \quad k \text{ s.t. } \lambda_{kj} \neq 0, \quad i = 0, \ldots, N-1 \tag{17.7.4}$$

But back to the discrete case: At an instant in time, if we know all the S_i's, we can compute all the rates a_j. Since rates are additive, the total rate at which *something* will happen is

$$a_{\text{tot}} = \sum_{j=0}^{M-1} a_j \tag{17.7.5}$$

Moreover, because the system is assumed to be "memoryless" (except for the S_j's) and "well-mixed," the probability distribution of times to this next occurrence of *some* reaction j must be exponentially distributed (like radioactive decay). Furthermore, given that *some* reaction occurs, it is easy to state what is the probability distribution of *which* reaction it is: It will be reaction j with probability a_j/a_{tot}.

This is all there is, conceptually, to stochastic simulation. The rest is just implementation details, including some clever tricks to speed up the calculation. The steps in the so-called *direct method* are

- From all the S_i's, compute all the a_j's and a_{tot}.
- Draw a random number U_1, uniform in $[0, 1]$, and compute the time τ to the next reaction by
$$\tau = \frac{1}{a_{\text{tot}}} \log\left(\frac{1}{U_1}\right) \tag{17.7.6}$$
(This generates an exponential distribution; cf. §7.3.)
- Draw a second uniform random number U_2 in $[0, 1]$ and find the smallest k such that
$$\sum_{j=0}^{k} a_j > a_{\text{tot}} U_2 \tag{17.7.7}$$
A value k will thus be chosen with probability a_k/a_{tot}.
- Increment the time t by τ.
- Update each S_i by adding the value ν_{ik}.
- Go back to the first step.

17.7.1 Speeding Up the Direct Method

We can speed up the direct method, first, by identifying all steps that are (naively) of $O(M)$ or $O(N)$ and finding ways to make them $O(1)$ (or maybe log); and second, by hand-crafting the inner loop of the program to have the fastest possible execution. The second of these tasks is very important and can make or break a stochastic simulation code's performance; but, unfortunately, it is very machine-, compiler-, and problem-dependent, so it is outside our scope here.

As for the first task, we first note that realistic reaction networks of any size almost always have very sparse reactant and state change matrices: Reactions generally involve only one or two reactants and produce at most a few products. Therefore, it is important to use some kind of sparse matrix structure for the matrices that occur.

When ν_{ij} is stored sparsely, for example, the updating step is reduced from $O(N)$ to $O(1)$.

Next, we note that most a_j's are unchanged after each reaction occurs. After a reaction k, for example, the only a_j's that need to be recomputed are those with reactants (inputs) that were changed by a nonzero entry in the kth column of the ν_{jk} matrix. A way to formalize this is by a *dependency graph* or *dependency matrix* G, whose component G_{jk} is nonzero only if reaction k changes a species that is an input to reaction j. With a moment of thought, you will figure out that the matrix G can be obtained by the logical matrix multiplication of λ^T and ν, namely

$$G_{ij} = \bigcup_k \lambda_{ki} \cap \nu_{kj} \tag{17.7.8}$$

where $\cup$ denotes logical-or, $\cap$ denotes logical-and, and the C convention of "true iff nonzero" is assumed. Now, after each reaction j, we only update the a_i's indicated by the jth column of G_{ij}. Of course we must also store G in a sparse format.

Finally, there is the question of how to speed up the choice of which reaction to update, equation (17.7.7), which can be at worst $O(M)$. Here there are two schools of thought. The one that we implement below, following advice in [3], takes advantage of the fact that for many, if not most, actual applications, a small number of reactions ($\ll M$) dominate the reaction rates. If we arrange the order of the a_j's in equation (17.7.7) with these dominant reactions first, then it can take, on average, only O(1) tests to select the next reaction. In [3], it is suggested to do preliminary runs to find which reactions dominate. We prefer the more transparent alternative, implemented below, of just letting frequent reactions adaptively bubble up in a priority list.

The other school of thought, called the *next reaction method* [2], is discussed separately, below. It cleverly changes $O(M)$ to something like $O(\log M)$, even in the most unfavorable case. However, the number of operations in the inner program loop is significantly larger than for the (optimized, as above) direct method. Which method is fastest is very likely problem- and implementation-dependent.

For the modest test case illustrated, namely the set of three reactions (17.7.1), most of the optimizations illustrated in the following code are unnecessary, and likely counterproductive. However, the intent is to be illustrative of what a code for larger problems would look like.

stochsim.h

```
struct Stochsim {
```
Object for stochastic simulation of a set of chemical reactions.
```
    VecDoub s;                                  Vector of species numbers.
    VecDoub a;                                  Vector of rates.
    MatDoub instate, outstate;
    NRvector<NRsparseCol> outchg, depend;       Sparse matrices ν_ij and G_ij
    VecInt pr;                                  Priority list.
    Doub t, asum;
    Ran ran;
    typedef Doub(Stochsim::*rateptr)();         Obscure C++ used to create a vec-
    rateptr *dispatch;                              tor dispatch of function point-
                                                    ers to the rate functions.
    // begin user section
```
Replace this section, using as a template the example (17.7.1) shown here, by the particulars of your reaction network. If you have a large number of reactions, you will want to generate the matrices instate and outstate externally, and pass them as globals (or read them here).
```
    static const Int mm=3;                      Set number of reactions.
    static const Int nn=4;                      Set number of species.
```

```
Doub k0,k1,k2;                                  Declare any rate constants needed.
Doub rate0() {return k0*s[0]*s[1];}             Your rate functions go here.
Doub rate1() {return k1*s[1]*s[2];}
Doub rate2() {return k2*s[2];}
void describereactions () {
```

You provide a function with this name that sets any constants that you have defined and sets the `instate` and `outstate` matrices to describe your reactions.

```
    k0 = 0.01;
    k1 = .1;
    k2 = 1.;
    Doub indat[] = {                            The reactant matrix λ_ij.
        1.,0.,0.,
        1.,1.,0.,
        0.,1.,1.,
        0.,0.,0.
    };
    instate = MatDoub(nn,mm,indat);
    Doub outdat[] = {                           The state change matrix ν_ij.
        -1.,0.,0.,
        1.,-1.,0.,
        0.,1.,-1.,
        0.,0.,1.
    };
    outstate = MatDoub(nn,mm,outdat);
    dispatch[0] = &Stochsim::rate0;             You must also point the dispatch ta-
    dispatch[1] = &Stochsim::rate1;                 ble entries to the correct rate
    dispatch[2] = &Stochsim::rate2;                 functions.
}
// end user section

Stochsim(VecDoub &sinit, Int seed=1)
```

Constructor. Input initial species numbers and an optional random seed.

```
: s(sinit), a(mm,0.), outchg(mm), depend(mm), pr(mm), t(0.),
asum(0.), ran(seed), dispatch(new rateptr[mm]) {
    Int i,j,k,d;
    describereactions();
    sparmatfill(outchg,outstate);
    MatDoub dep(mm,mm);
    for (i=0;i<mm;i++) for (j=0;j<mm;j++) {     Logical matrix multiply calculates the
        d = 0;                                      dependency matrix.
        for (k=0;k<nn;k++) d = d || (instate[k][i] && outstate[k][j]);
        dep[i][j] = d;
    }
    sparmatfill(depend,dep);
    for (i=0;i<mm;i++) {                        Calculate all initial rates.
        pr[i] = i;
        a[i] = (this->*dispatch[i])();
        asum += a[i];
    }
}
~Stochsim() {delete [] dispatch;}

Doub step() {
```

Take a single stochastic step (one reaction) and return the new time.

```
    Int i,n,m,k=0;
    Doub tau,atarg,sum,anew;
    if (asum == 0.) {t *= 2.; return t;}        Rare: All reactions have stopped ex-
    tau = -log(ran.doub())/asum;                    actly, so double the time until
    atarg = ran.doub()*asum;                        the user notices!
    sum = a[pr[0]];
    while (sum < atarg) sum += a[pr[++k]];      Equation (17.7.7).
    m = pr[k];
    if (k > 0) SWAP(pr[k],pr[k-1]);             Move reaction up on the priority list.
    if (k == mm-1) asum = sum;                  Free update of asum fixes accumu-
                                                    lated roundoff.
```

```
        n = outchg[m].nvals;
        for (i=0;i<n;i++) {                          Apply state change vector.
            k = outchg[m].row_ind[i];
            s[k] += outchg[m].val[i];
        }
        n = depend[m].nvals;
        for (i=0;i<n;i++) {                          Recalculate rates required by depen-
            k = depend[m].row_ind[i];                    dency matrix.
            anew = (this->*dispatch[k])();
            asum += (anew - a[k]);
            a[k] = anew;
        }
        if (t*asum < 0.1)                            Rare: Rates heading toward zero.
            for (asum=0.,i=0;i<mm;i++) asum += a[i];     Better recalculate asum.
        return (t += tau);
    }
};
```

Note that `Stochsim` uses some arcane C++ syntax ("array of pointers to member functions") in connection with the identifier `dispatch`. The underlying idea is simple, and important: We want to jump directly to the appropriate user-supplied rate function, as directed by an integer index. There are various ways of coding this, but what you *don't* want to have is a long chain of `if` tests that would be $O(M)$ instead of $O(1)$. (Perhaps we should believe that C's `switch` statement is always properly implemented by compilers as a fast table dispatch, but we don't.)

The utility routine that constructs a sparse matrix out of a full one is this (cf. §2.7):

```
void sparmatfill(NRvector<NRsparseCol> &sparmat, MatDoub &fullmat) {                stochsim.h
Utility that fills a sparse matrix from a full one. See §2.7.
    Int n,m,nz,nn=fullmat.nrows(),mm=fullmat.ncols();
    if (sparmat.size() != mm) throw("bad sizes");
    for (m=0;m<mm;m++) {
        for (nz=n=0;n<nn;n++) if (fullmat[n][m]) nz++;
        sparmat[m].resize(nn,nz);
        for (nz=n=0;n<nn;n++) if (fullmat[n][m]) {
            sparmat[m].row_ind[nz] = n;
            sparmat[m].val[nz++] = fullmat[n][m];
        }
    }
}
```

As a cultural note, the system (17.7.1) is not just any old chemical reaction network, but is actually a form of the *Lotka-Volterra* equation, discovered independently by Alfred J. Lotka and Vito Volterra in 1925–1926. In fact, it's not originally a chemical reaction network at all, but a set of relationships intended to model predator-prey relationships. The first equation says, roughly, that rabbits (X) eat grass (A) to produce more rabbits. The second says that foxes (Y) eat rabbits (X) to produce more foxes. The third says that foxes don't live forever. (For some reason, rabbits do live forever in this model, unless they are eaten by foxes.)

Figure 17.7.1 shows an example of the system's evolution, starting with initial conditions $A = 150$, $X = Y = 10$, and $B = 0$. One sees two cycles of prey population growth, with predator growth following, and then a collapse of both populations. After the second cycle, by a fluctuation, the predator population goes to zero, from which it cannot recover. At the end of the evolution shown, the prey population is starting to recover; but it is not a happy ending, because, by now, the food supply is running out. The world of stochastic simulation is a harsh one. Stochastic ef-

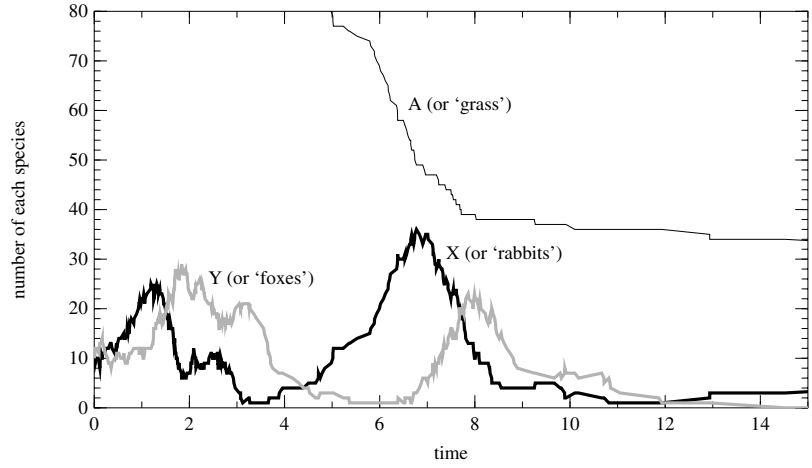


Figure 17.7.1. Evolution of the reaction network (17.7.1). This network evolves by the Lotka-Volterra equations, originally developed as a model for predator-prey interactions. Stochastic effects are important; with different random seeds, different time histories occur.

fects are genuinely dominant in this example: Exactly the same equations and initial conditions, but with a different random seed, give entirely different evolutions.

17.7.2 Next Reaction Method

The *next reaction method* [2] starts by computing not a single reaction time, equation (17.7.6), but rather a separate next reaction time for each reaction j,

$$\tau_j = \frac{1}{a_j} \log\left(\frac{1}{U_j}\right) \tag{17.7.9}$$

where the U_j's are independent uniform random deviates in $[0, 1]$. These times are all stored in a heap (see §8.3), so that the smallest value can be easily accessed at the top of the heap (call it k). The following steps are now repeatedly executed:

- Do reaction k and update the affected S_i's (using the matrix ν_{ik}). Increment time t by τ_k.
- Compute a next time for reaction k (using equation 17.7.9 and adding t) and store it on the heap.
- For every affected reaction j (as determined by a nonzero entry G_{jk}), correct its stored next time by the formula

$$\tau_j \leftarrow \frac{a_{j,\text{old}}}{a_{j,\text{new}}}(\tau_j - t) + t \tag{17.7.10}$$

  This is called *time reuse*. In effect, it reuses the random deviate U_j that originally generated τ_j, but it corrects the resulting time prediction for an intermediate step function change in a_j. Sounds dodgy, we know, but it is probabilistically sound.
- Get the heap back into order by bubbling elements up or down as required. This is where the complexity of the inner loop gets increased, to as much as $O(\log M)$.

Unquestionably, one can construct reaction networks for which the next reaction method is considerably faster than the optimized direct method. However, networks dominated by a small number of fast reactions are so common in practice that this performance advantage should not be assumed a priori [3].

17.7.3 Practical Advice

Don't ever use a stochastic simulation method — of any flavor — unless your problem is genuinely stochastic. Instead, use the deterministic rate equations (17.7.4) with a good stiff equation solver like `StepperSie` in §17.5. Such solvers are not limited by the rate of the fastest reaction, and will frequently be orders of magnitude faster than any stochastic method. (We are reliably informed that an unconscionable number of CPU hours are wasted by misguided researchers who think that the stochastic simulation method is an all-purpose tool for reaction networks.)

Just to show you how easy this is, here is how you would do the Lotka-Volterra problem (17.7.2) by integrating the equations directly. First encode the right-hand side $\mathbf{f}$ and the Jacobian of the right-hand side in a structure. (The (i, j) element of the Jacobian is $\partial f_i / \partial y_j$.)

```
struct rhs {
    Doub k0,k1,k2;
    rhs(Doub kk0, Doub kk1, Doub kk2) : k0(kk0),k1(kk1),k2(kk2) {}
    void operator() (const Doub x, VecDoub_I &y, VecDoub_O &dydx) {
        dydx[0]= -k0*y[0]*y[1];
        dydx[1]= k0*y[0]*y[1]-k1*y[1]*y[2];
        dydx[2]= k1*y[1]*y[2]-k2*y[2];
        dydx[3]= k2*y[2];
    }
    void jacobian(const Doub x, VecDoub_I &y, VecDoub_O &dfdx,
            MatDoub_O &dfdy) {
        Int n=y.size();
        for (Int i=0;i<n;i++) dfdx[i]=0.0;
        dfdy[0][0] = -k0*y[1];
        dfdy[0][1] = -k0*y[0];
        dfdy[0][2] = 0.0;
        dfdy[0][3] = 0.0;
        dfdy[1][0] = k0*y[1];
        dfdy[1][1] = k0*y[0]-k1*y[2];
        dfdy[1][2] = -k1*y[1];
        dfdy[1][3] = 0.0;
        dfdy[2][0] = 0.0;
        dfdy[2][1] = k1*y[2];
        dfdy[2][2] = k1*y[1]-k2;
        dfdy[2][3] = 0.0;
        dfdy[3][0] = 0.0;
        dfdy[3][1] = 0.0;
        dfdy[3][2] = k2;
        dfdy[3][3] = 0.0;
    }
};
```

Next set the parameters for `Odeint`, for example

```
const Int n=4;
Doub rtol=1.0e-7,atol=1.0e-4*rtol,h1=1.0e-6,hmin=0.0,x1=0.0,x2=15.0;
VecDoub ystart(n);
ystart[0]=150.0;
ystart[1]=10.0;
```

```
    ystart[2]=10.0;
    ystart[3]=0.0;
    Output out(100);                          Output at 100 uniform points
    rhs d(0.01,0.1,1.0);                      Declare d as a rhs object.
    Odeint<StepperSIE<rhs> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
    ode.integrate();
```

Note how the values of k_0, k_1, and k_2 are passed as arguments in the constructor call that declares `d`. These particular values don't make the system of equations particularly stiff, so you could use a standard integrator. However, this is not true in general for real-world examples.

The output, which is equally spaced, can be printed by statements like

```
for (Int i=0;i<out.count;i++)
    cout << out.xsave[i] << " " << out.ysave[0][i] << " " <<
        out.ysave[1][i] << " " << out.ysave[2][i] << endl;
```

If your network's fastest reactions are not stochastic, but there are some slower reactions where stochastic effects are important, then look into so-called hybrid methods (e.g., [4]).

CITED REFERENCES AND FURTHER READING:

Gillespie, D.T. 1976, "A General Method for Numerically Simulating the Stochastic Time Evolution of Coupled Chemical Reactions," *Journal of Computational Physics*, vol. 11, pp. 403–434.[1]

Gibson, M.A., and Bruck, J. 2000, "Efficient Exact Stochastic Simulation of Chemical Systems with Many Species and Many Channels," *Journal of Physical Chemistry A*, vol. 104, pp. 1876–1889.[2]

Cao, Y., Li, H., and Petzold, L. 2004, "Efficient Formulation of the Stochastic Simulation Algorithm for Chemically Reacting Systems," *Journal of Chemical Physics*, vol. 121, pp. 4059–4067.[3]

Salis, H., and Kaznessis, Y. 2005, "Accurate Hybrid Stochastic Simulation of a System of Coupled Chemical or Biochemical Reactions," *Journal of Chemical Physics*, vol. 122, art. 054103.[4]